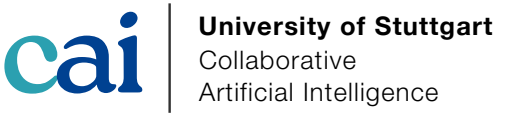


Machine Perception and Learning

for Collaborative Intelligent Systems

Prof. Dr. Andreas Bulling

WS 2025/2026

Collaborative Artificial Intelligence Group, University of Stuttgart

www.collaborative-ai.org

Recurrent Neural Networks

- Backpropagation Through Time (BPTT)
- An Example Task
- Long Short-Term Memory (LSTM)

Transformers

- Embeddings
- Attention Mechanisms
- Attention is all you need
- BERT

Transformers

Embeddings

Embeddings – Motivation

- Many machine learning algorithms cannot work with categorical data (both input and output)
- Categorical data has to be converted to numeric values first
- Naïve approach: Convert to integers

- Many machine learning algorithms cannot work with categorical data (both input and output)
- Categorical data has to be converted to numeric values first
- Naïve approach: Convert to integers
- **Problem**: Doesn't work if there is no ordinal relationship between values (e.g. "cat" == 1, "dog" == 2, "kitten" == 3)
- Better solution: Word embeddings

Core idea: Individual categorical attributes (e.g. words or labels) are represented as real-valued vectors in a predefined vector space

How to convert words to numbers (vectors)?

- **One-hot encoding**
- **Learned embeddings**
- ”You shall know a word by the company it keeps” [Firth, 1957] (Distribution hypothesis)

Encode words as binary vectors (**sparse** word representation):

Cat	Dog	Rabbit	Snake	
1	0	0	0	…
0	1	0	0	…
0	0	1	0	…
0	0	0	1	…
…	…	…	…	…

Cat = [1,0,0,0, …]

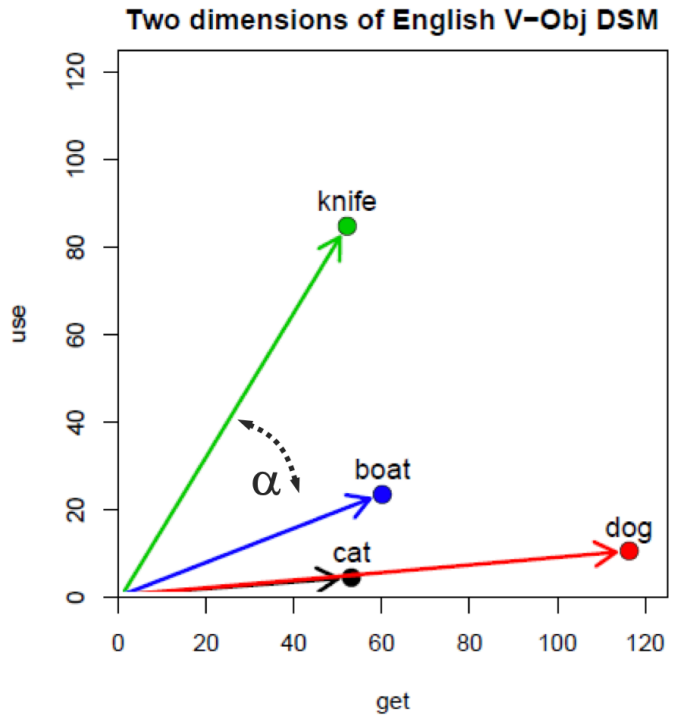
Dog = [0,1,0,0, …]

Rabbit = [0,0,1,0, …]

Snake = [0,0,0,1, …]

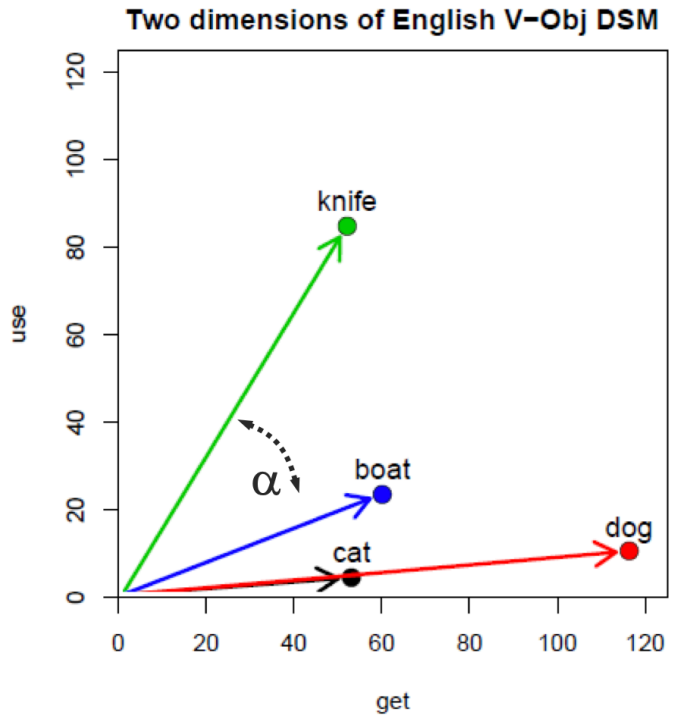
What's the limitation of one-hot encoding?

- Learned real-valued vectors for each word (**dense** word representation)
- Joint vector space with distance measure α
  - Angle is more important than distance



Source: Louis-P. Morency, CMU

- Learned real-valued vectors for each word (**dense** word representation)
- Joint vector space with distance measure α
  - Angle is more important than distance
  - Can use algebra: $\vec{king} - \vec{man} + \vec{woman} = \vec{queen}$



Source: Louis-P. Morency, CMU

Embeddings – How do we learn such embeddings?

- Continuous Bag-of-Words (CBOW)
  - Takes a context window around a focus word
  - Predicts the focus word

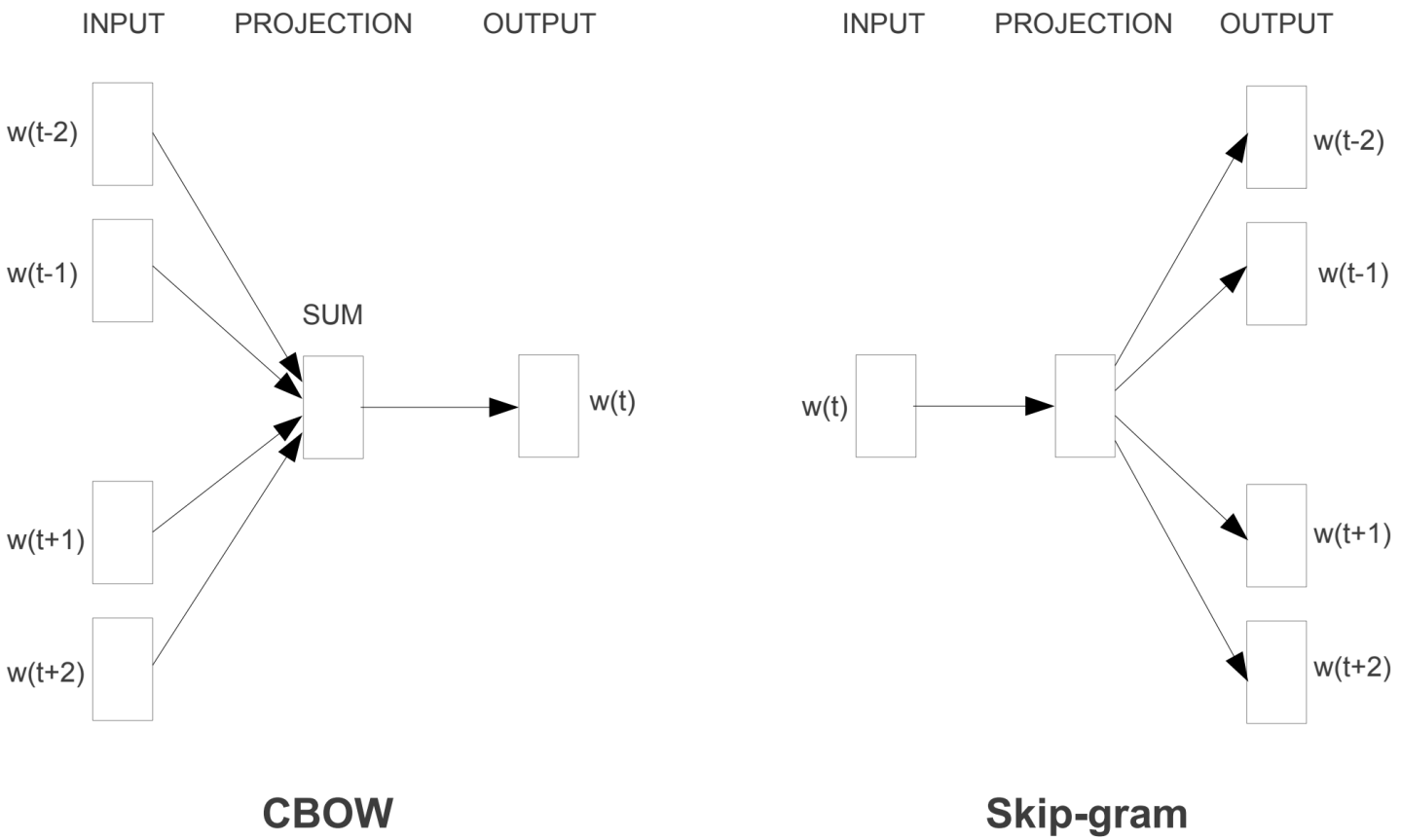
Embeddings – How do we learn such embeddings?

- Continuous Bag-of-Words (CBOW)
  - Takes a context window around a focus word
  - Predicts the focus word
- Skip-gram model
  - Takes a focus word
  - Predicts the context window

Embeddings – How do we learn such embeddings?

- Continuous Bag-of-Words (CBOW)
  - Takes a context window around a focus word
  - Predicts the focus word
- Skip-gram model
  - Takes a focus word
  - Predicts the context window
- Statistical
  - Use probabilities to determine word co-occurrence

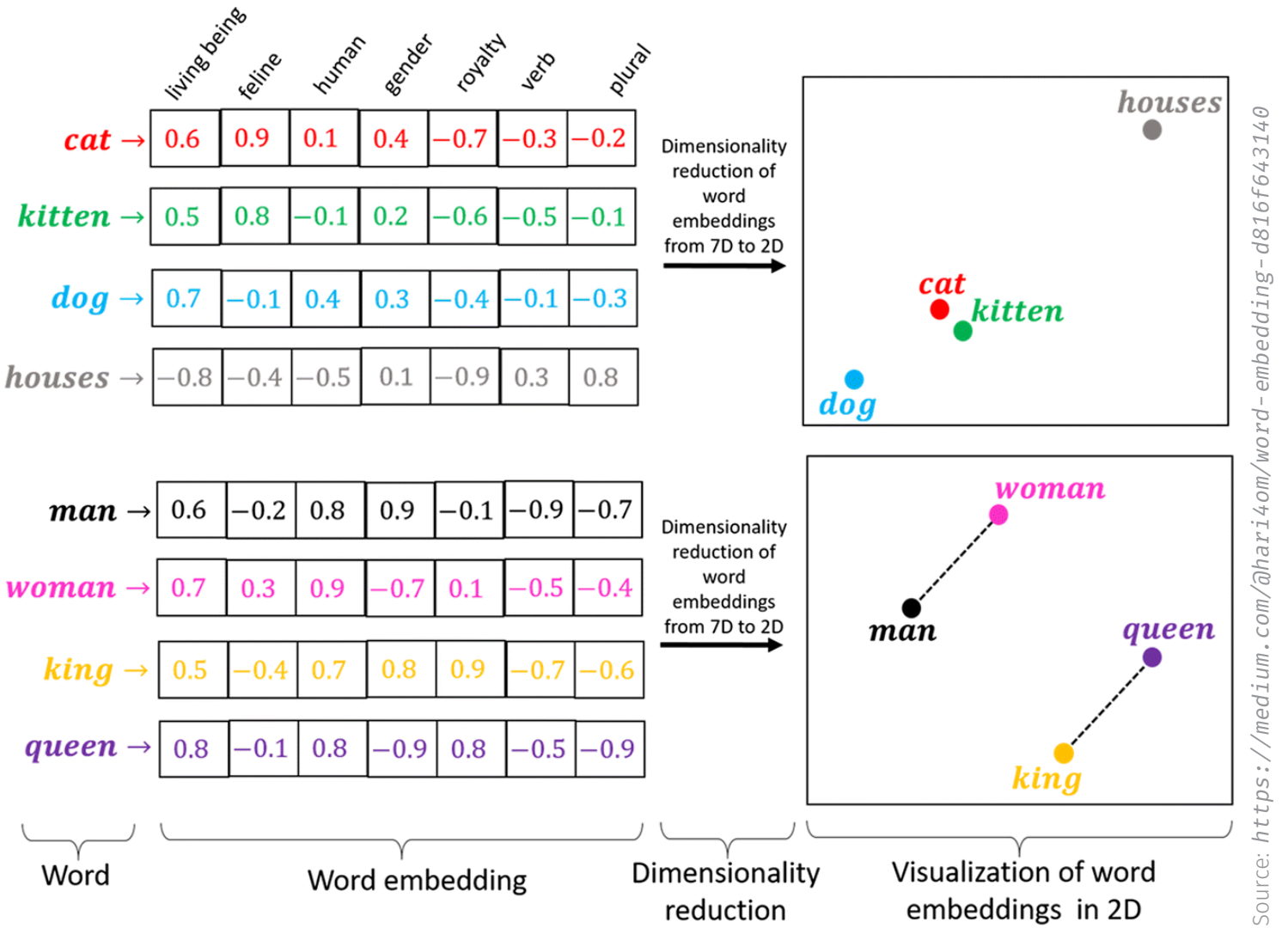
Embeddings – Continuous Bag-of-Words vs. Skip-gram



Source: Mikolov et al. [2013b]

- Word2Vec (2013)
  - Trained on the Google dataset, 100 billion words
  - Uses local window for context
  - Can be CBOW or Skip-gram

Slide source: [Mikolov et al., 2013a]

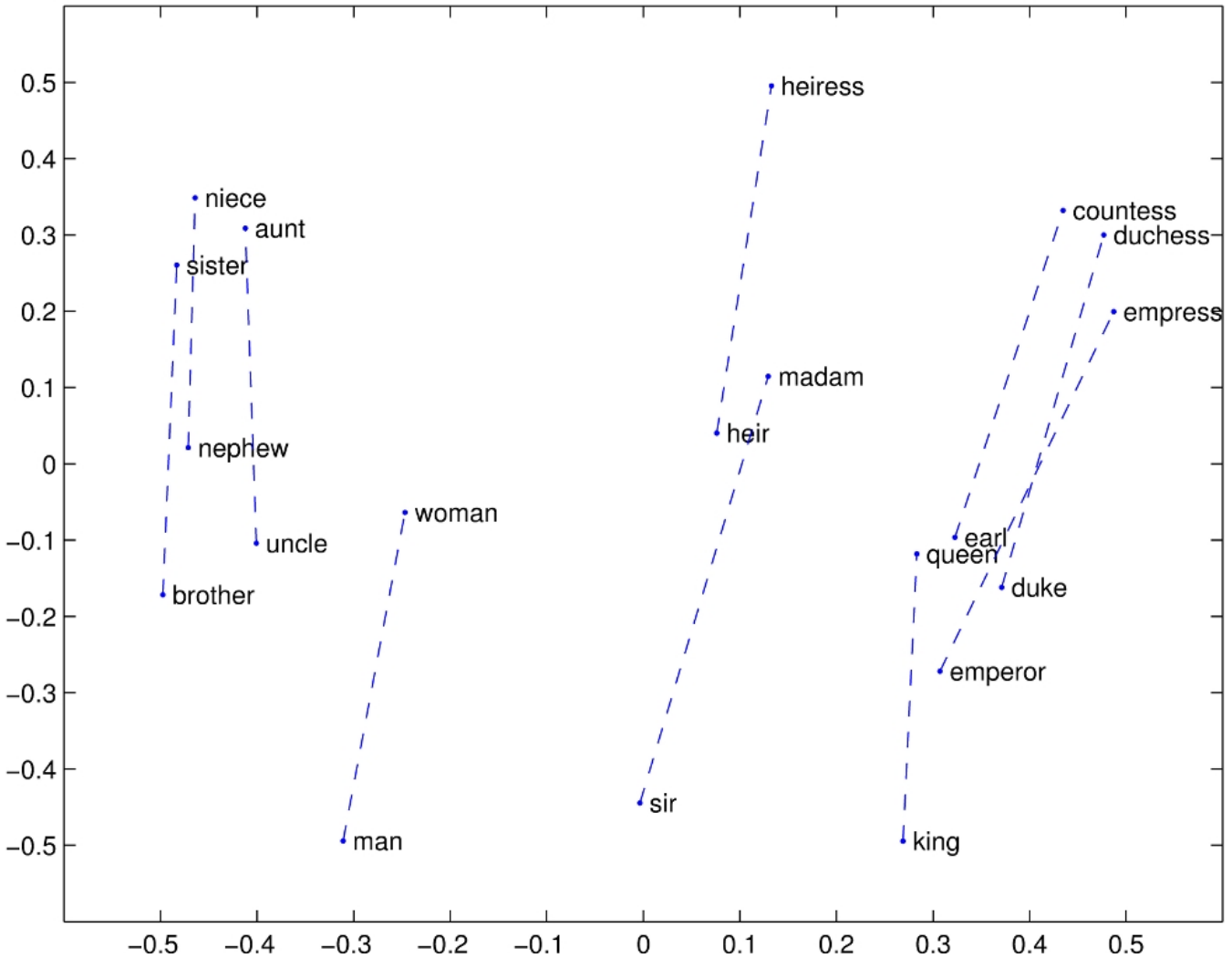


Source: https://medium.com/@hari4om/word-embedding-d816f643140

- GloVe (2014) - uses whole corpus for context
  - Uses statistics to encode probabilities of words co-occurring
  - Based on the ratio of co-occurrence probabilities
  - Weighted least squares regression model

Slide source: [Pennington et al., 2014]

what did you find between embeddings?

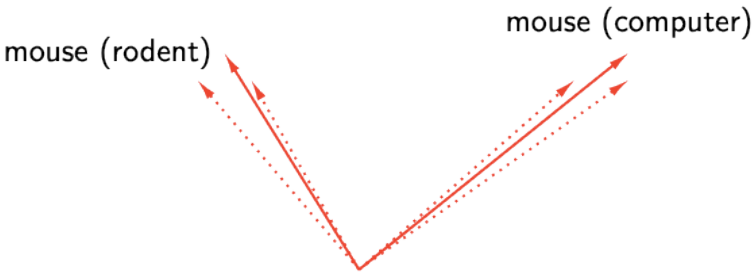


Source: https://nlp.stanford.edu/projects/glove/

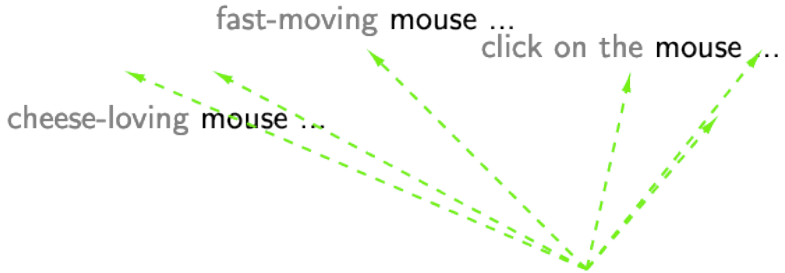
- **Non-contextual** (Word2Vec, GloVe)
  - Only one representation is learnt for each word no matter the context/sentence
  - Example: ”I **left** my phone on the **left** side of the table.”
  - ”left” has two different meanings in the sentence but does not have two different representations in the embedding space

- **Non-contextual** (Word2Vec, GloVe)
  - Only one representation is learnt for each word no matter the context/sentence
  - Example: ”I **left** my phone on the **left** side of the table.”
  - ”left” has two different meanings in the sentence but does not have two different representations in the embedding space
- **Contextual** (BERT, ELMo, GPT, etc.)
  - Takes into account the context, i.e. different embeddings for the same word depending on context

Embeddings – How embeddings are contextual?



Source: *http://ai.stanford.edu/blog/contextual/*



Source: *http://ai.stanford.edu/blog/contextual/*

- Compares BERT, ELMo, and GPT-2
- Three new measures of contextuality: self- & intra-sentence similarity, maximum explainable variance (MEV)
  - MEV: proportion of variance in a word's representation that can be explained by its first principal component

Slide source: [Ethayarajh, 2019]

Embeddings – How embeddings are contextual: Findings

- Representations of words are anisotropic: they occupy a narrow cone in the embedding space
- Upper layers produce more context-specific representations than lower layers
- Models contextualise words (very) differently
- Less than 5% of the variance of a word's contextualised representations can be explained by a static embedding
- Static embeddings created from the first principal component in a lower layer can outperform GloVe and FastText

Transformers

Attention Mechanisms

RNNs have several weaknesses:

- Slow training speed
- Long-distance temporal dependencies (vanishing gradients)
- Difficulty dealing with long input sequences (e.g. loss on Wikipedia)
- Also difficulties with longer input sequences than those seen during training

How to address these limitations?

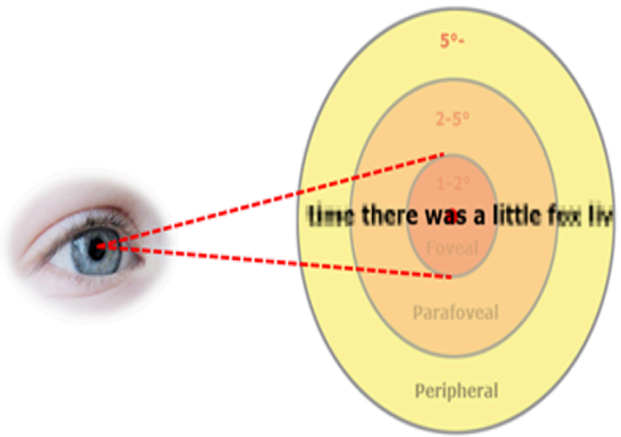
RNNs have several weaknesses:

- Slow training speed
- Long-distance temporal dependencies (vanishing gradients)
- Difficulty dealing with long input sequences (e.g. loss on Wikipedia)
- Also difficulties with longer input sequences than those seen during training

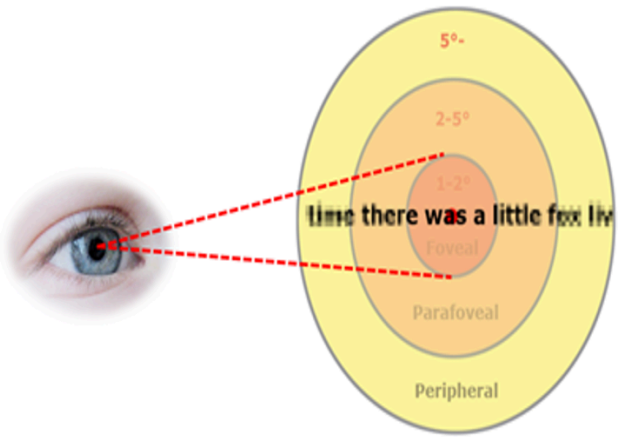
How to address these limitations? With **attention!**

- Neural attention mechanisms are inspired by human attention
- Humans only perceive with high acuity within about 2 degrees of visual angle (foveal vision)
  - We don't perceive a whole image at once but instead **focus** on different parts sequentially → scanpaths composed of eye fixations
  - "Attention is the flexible control of limited computational resources" [Lindsay, 2020]

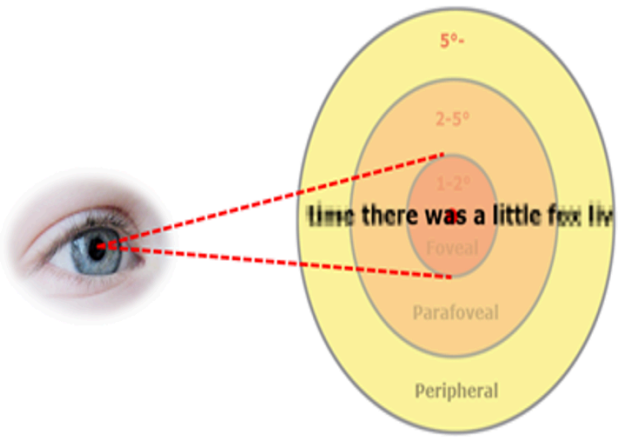
- Alertness or vigilance to surroundings



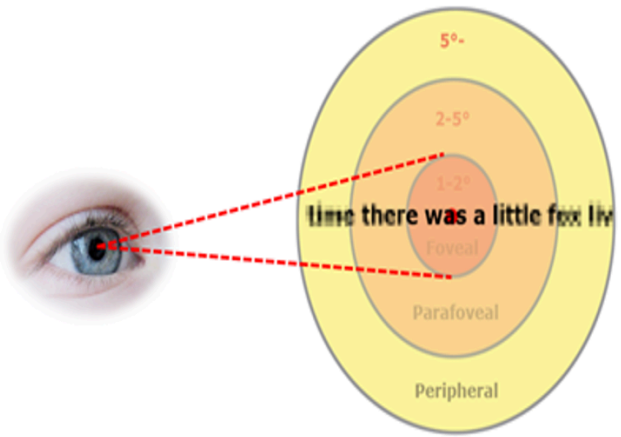
- Alertness or vigilance to surroundings
- Can be selective to different sensory inputs



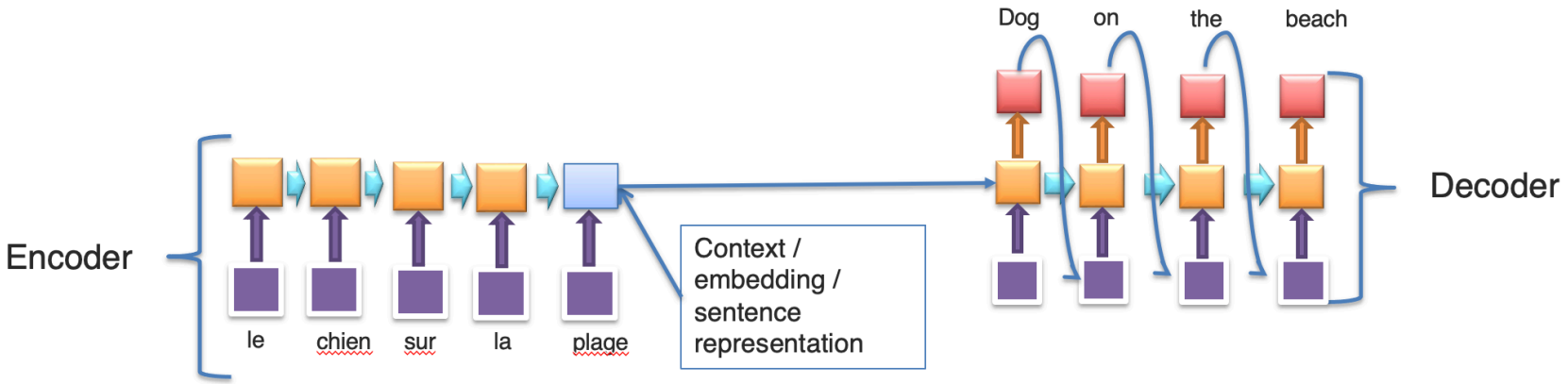
- Alertness or vigilance to surroundings
- Can be selective to different sensory inputs
- Saliency = “attracting attention”



- Alertness or vigilance to surroundings
- Can be selective to different sensory inputs
- Saliency = “attracting attention”
- Neurons associated with the stimulus will fire differently (e.g. more in sync)



Quick reminder: **Many-to-many** architecture, e.g. as used for machine translation



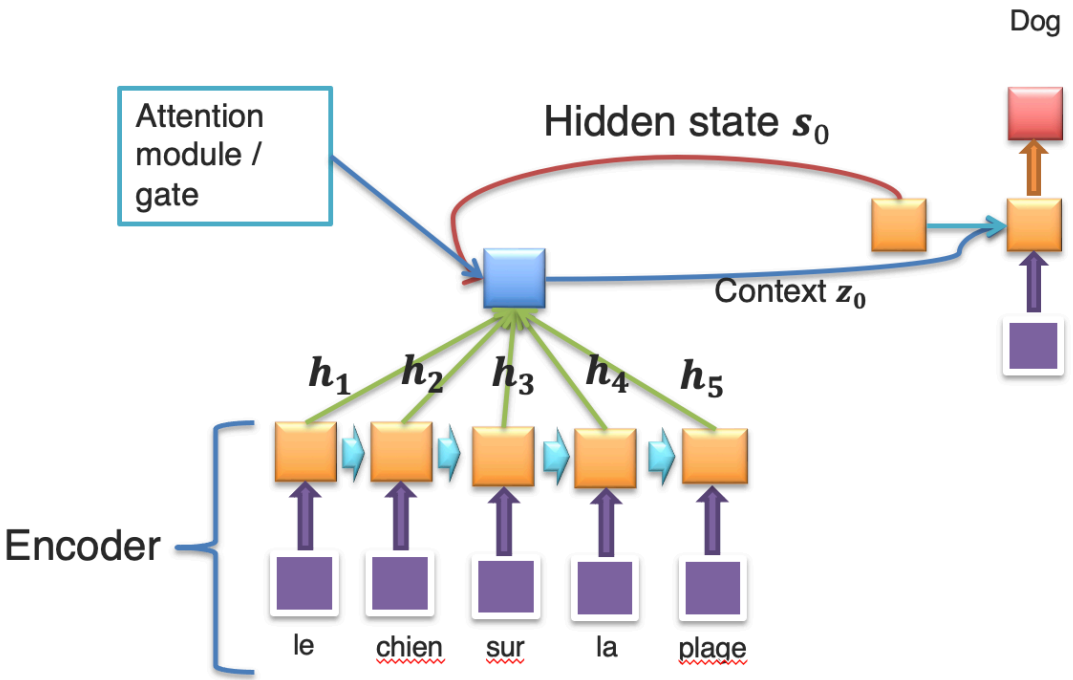
Source: Louis-P. Morency, CMU

What's the problem with this?

What happens when the sentences are very long?

Slide source: [Bahdanau et al., 2015]

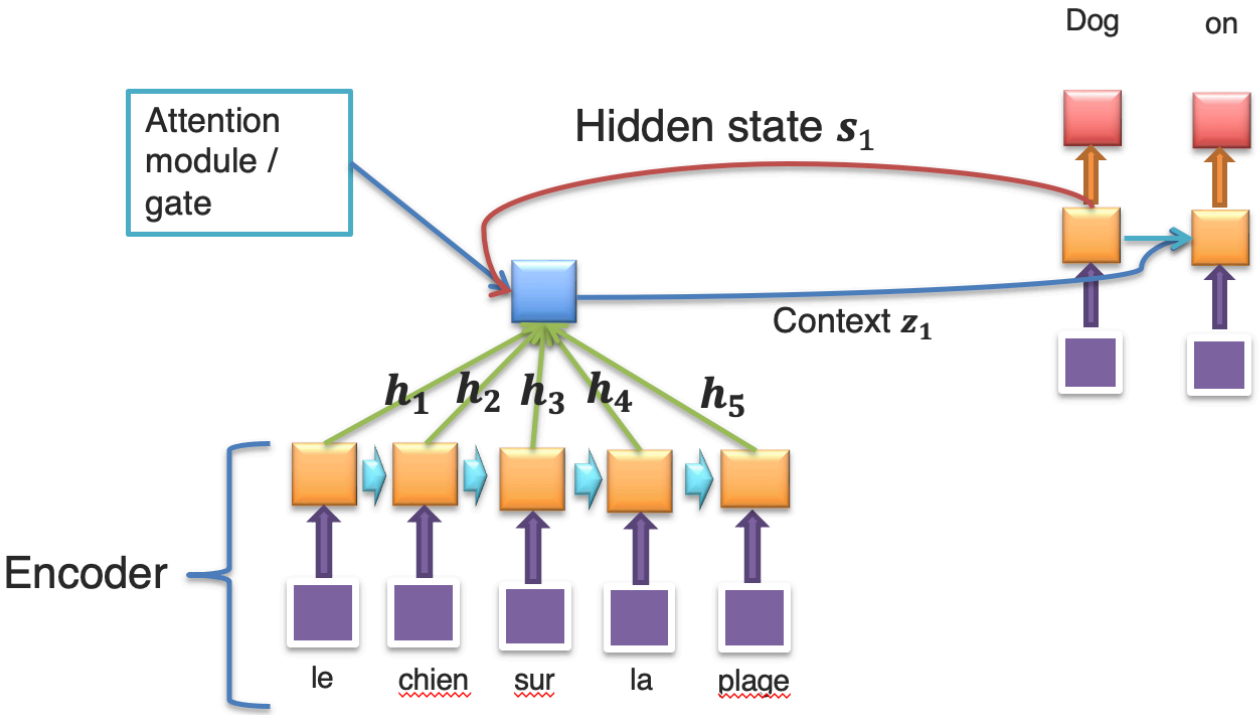
A new intermediate hidden state is computed for each decoding iteration:



Source: Louis-P. Morency

Slide source: [Bahdanau et al., 2015]

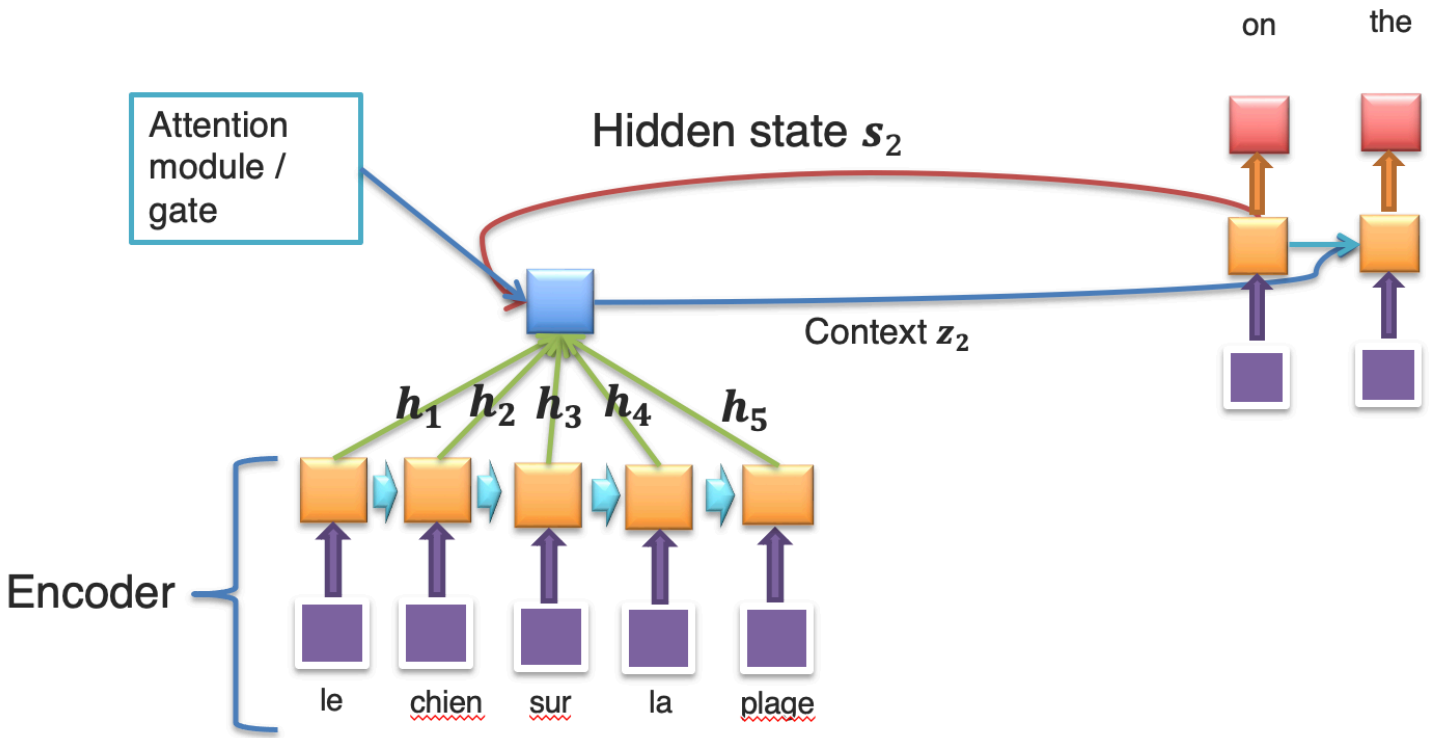
A new intermediate hidden state is computed for each decoding iteration:



Source: Louis-P. Morency, CMU

Slide source: [Bahdanau et al., 2015]

A new intermediate hidden state is computed for each decoding iteration:



Source: Louis-P. Morency, CMU

Slide source: [Bahdanau et al., 2015]

- **Before**: $p(y_i|y_1, ..., y_{i-1}, x) = g(y_{i-1}, s_i, z)$
  - where $z = h_T$ is the last encoder state and s_i is the state of the decoder

Slide source: Louis-Philippe Morency, CMU

- **Before**: $p(y_i|y_1, ..., y_{i-1}, x) = g(y_{i-1}, s_i, z)$
  - where $z = h_T$ is the last encoder state and s_i is the state of the decoder
- **Now**: $p(y_i|y_1, ..., y_{i-1}, x) = g(y_{i-1}, s_i, z_i)$
  - Attention "gate"
  - Different context z_i is used at each time step!
  - $z_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$ where α_{ij} is the scalar attention weight for word j at generation step i (expectation of the context)

How to determine α_{ij}?

How to determine α_{ij}?

$$\alpha_{ij} = \frac{exp(e_{ij})}{\sum_{k=1}^{T_x} exp(e_{ik})}$$

softmax to make sure they sum to 1

Slide source: Louis-Philippe Morency, CMU

How to determine α_{ij}?

$$\alpha_{ij} = \frac{exp(e_{ij})}{\sum_{k=1}^{T_x} exp(e_{ik})}$$

softmax to make sure they sum to 1

where

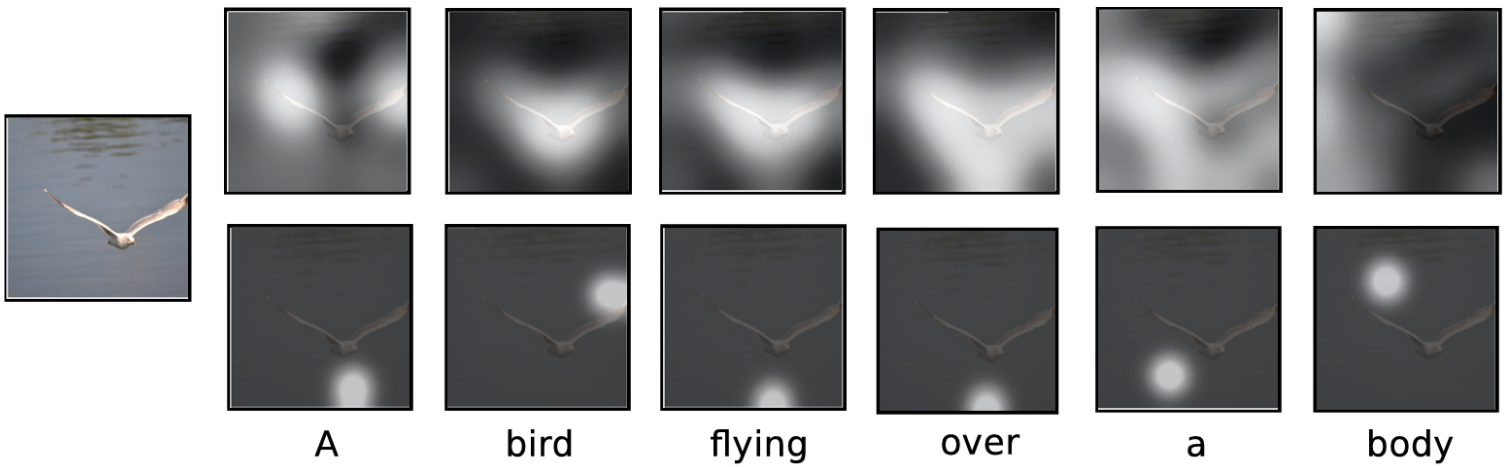
$$e_{ij} = v^T \sigma(W s_{i-1} + U h_j)$$

$\rightarrow$ feed-forward network that predicts, given the state of the decoder s_{i-1}, how important the current encoding h_j is

v, W, U are its learnable weights

- **Soft attention**
  - Deterministic, differentiable
  - Takes expectation of context vector (weighted average)
  - Multiply attention map over feature map; ”looks” everywhere
- **Hard attention** (e.g. [Mnih et al., 2014; Ba et al., 2014])
  - Stochastic
  - Samples attention locations with probabilities
  - ”Looks” only at one area at a time
  - As such, closely resembles human visual attention

Soft vs Hard Attention

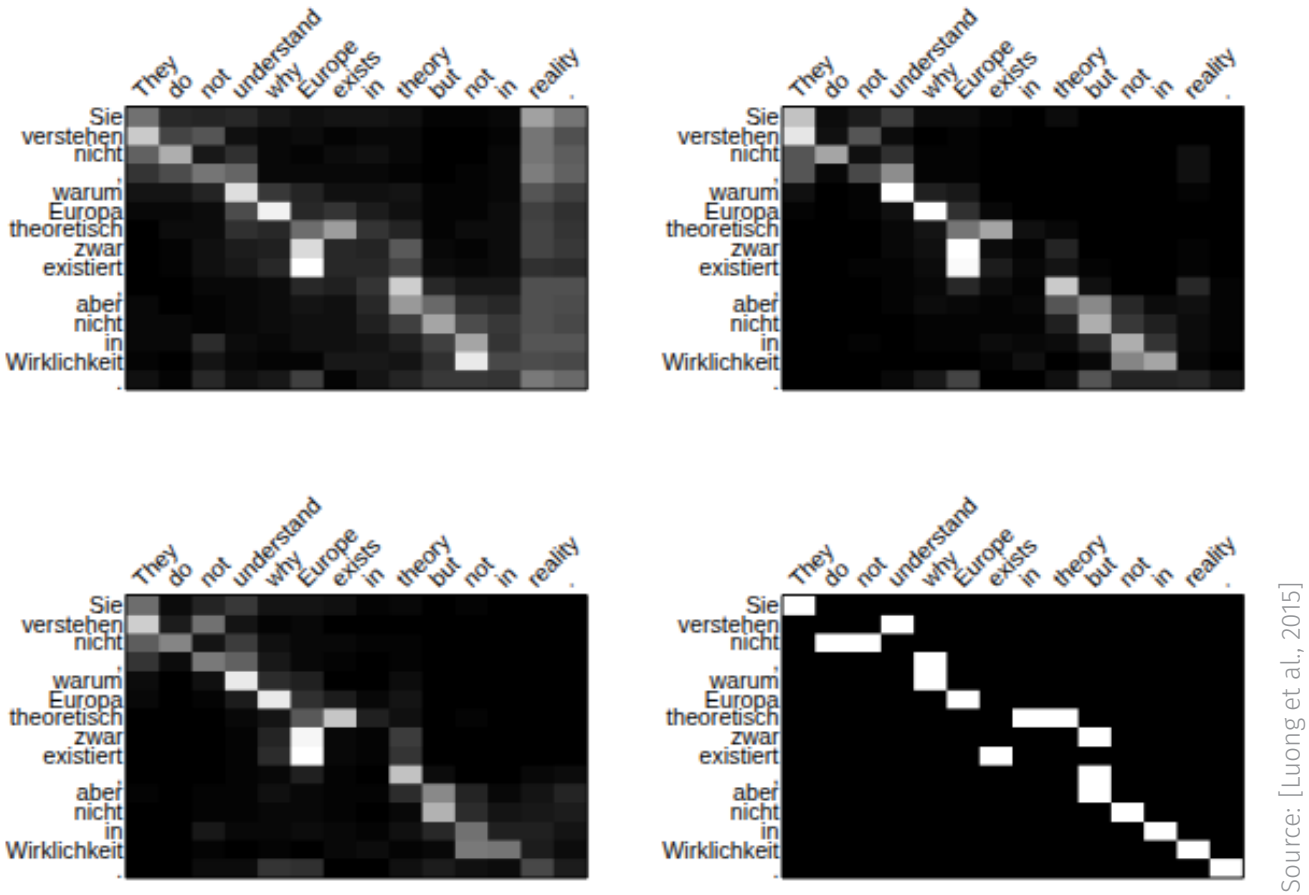


Source: [Xu et al., 2015]

- Global attention
  - All hidden states are considered to create context vector
  - Expensive and sometimes impractical
- Local attention
  - Focuses on a subset of hidden states

Slide source: [Luong et al., 2015]

Global vs. Local Attention



Source: [Luong et al., 2015]

Attention Mechanisms – Advantages

- Allows for more flexibility
  - e.g. different input sizes

Attention Mechanisms – Advantages

- Allows for more flexibility
  - e.g. different input sizes
- Gives better performance, particularly for long inputs

- Allows for more flexibility
  - e.g. different input sizes
- Gives better performance, particularly for long inputs
- Allows for more interpretability
  - Area of focus (on images or text) is observable

- Solves some of the long dependency problems
- Still slow to train

Question: How to address this?

- Solves some of the long dependency problems
- Still slow to train

Question: How to address this?

Answer: Use attention only! That means...

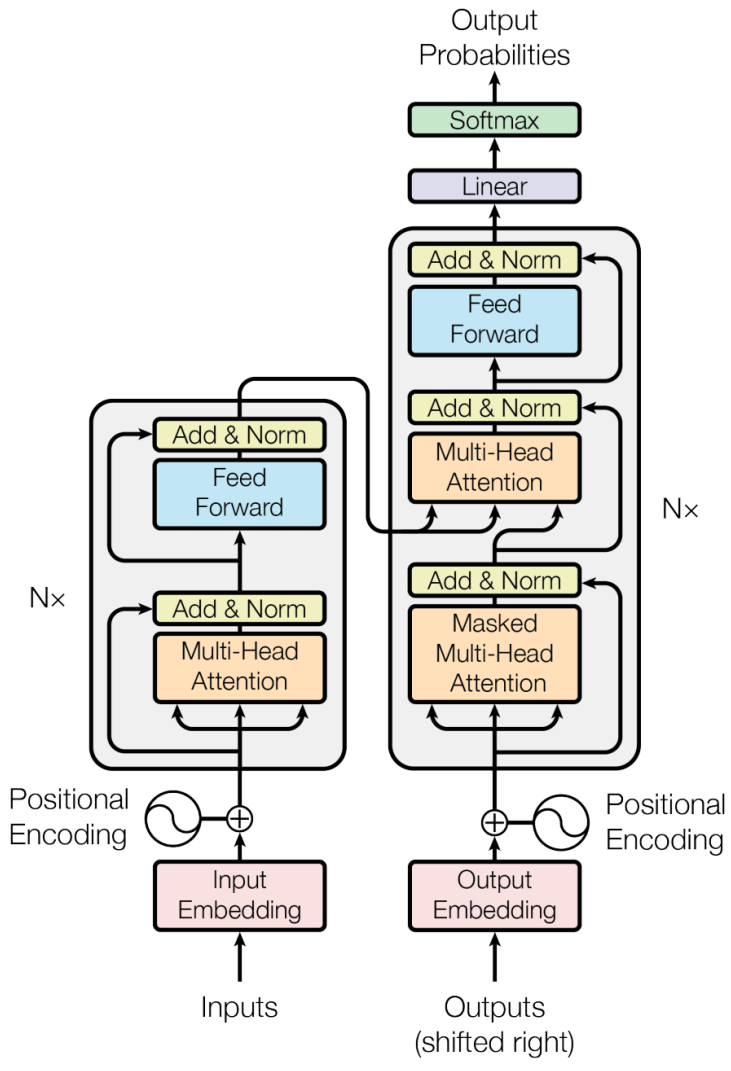
- ... remove recurrence/sequence
- ... use several attention layers

Transformers

Attention is all you need

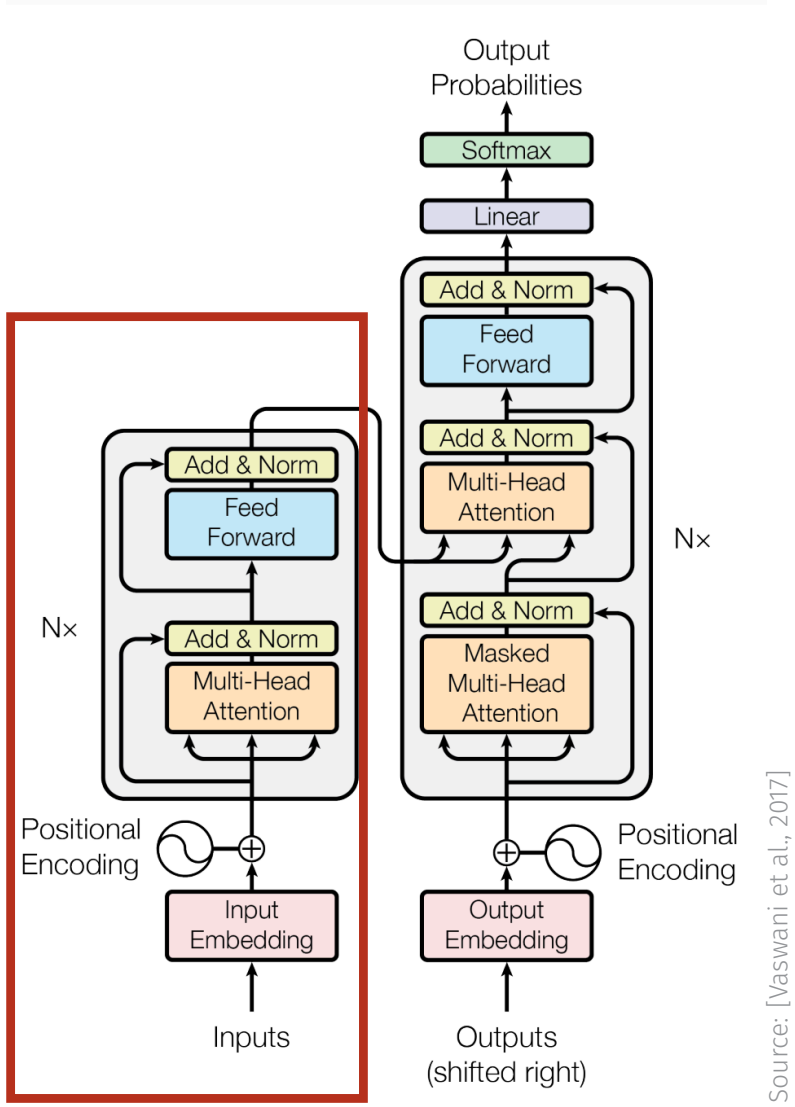
- NeurIPS 17': Attention is all you need [Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, and Polosukhin, 2017]
- 202k+ citations
- The Annotated Transformer in Notebook:
  http://nlp.seas.harvard.edu/2018/04/03/attention.html

- Only uses attention to identify important parts of the input
- Combining attention with parallelisation



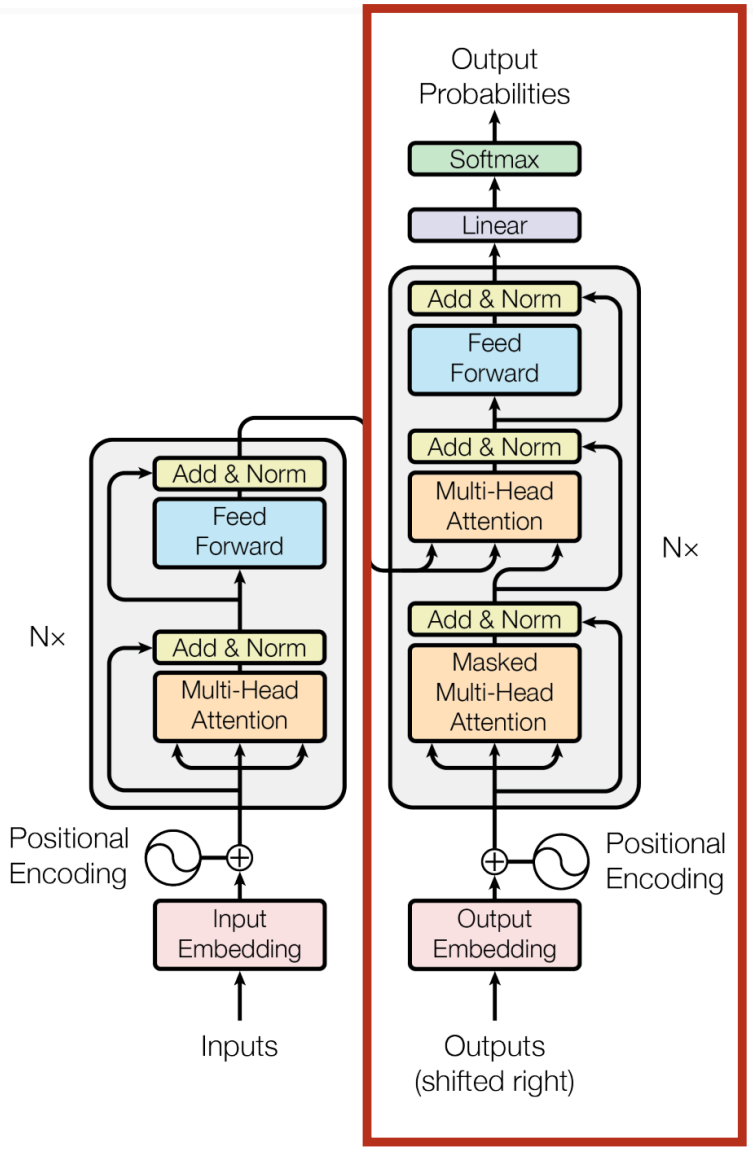
Source: [Vaswani et al., 2017]

- Several encoder modules stacked on top of each other (e.g. 6)



Source: [Vaswani et al., 2017]

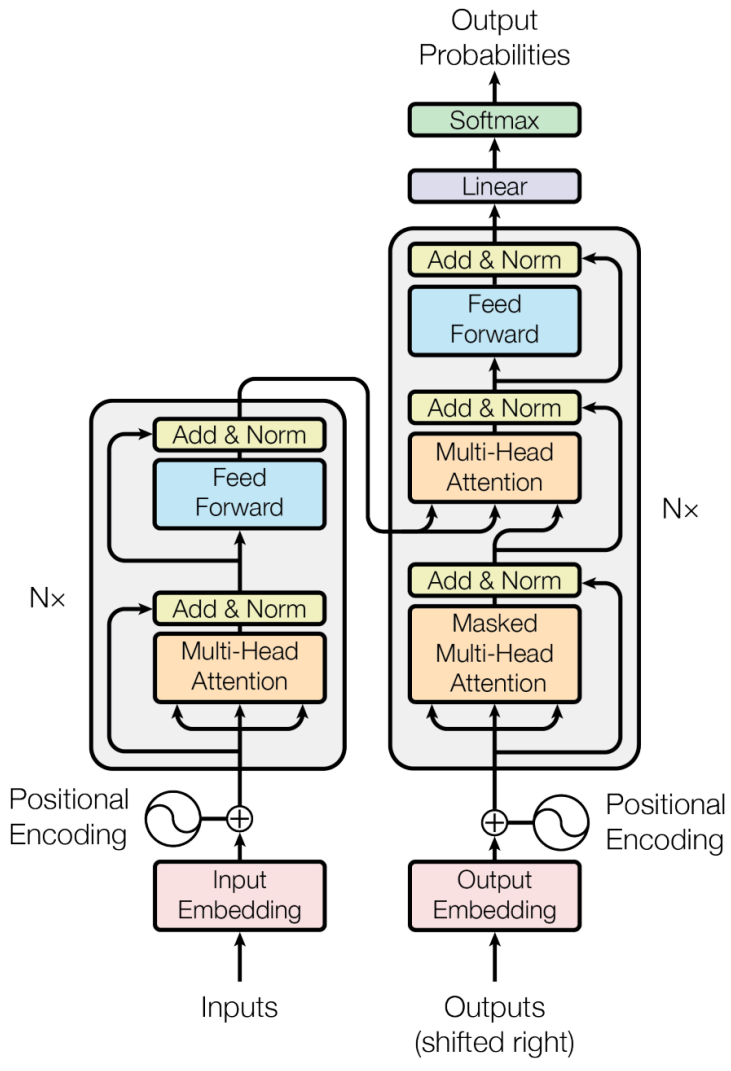
- Several decoder modules stacked on top of each other (e.g. 6)
- No recurrence, no autoregressive models



Source: [Vaswani et al., 2017]

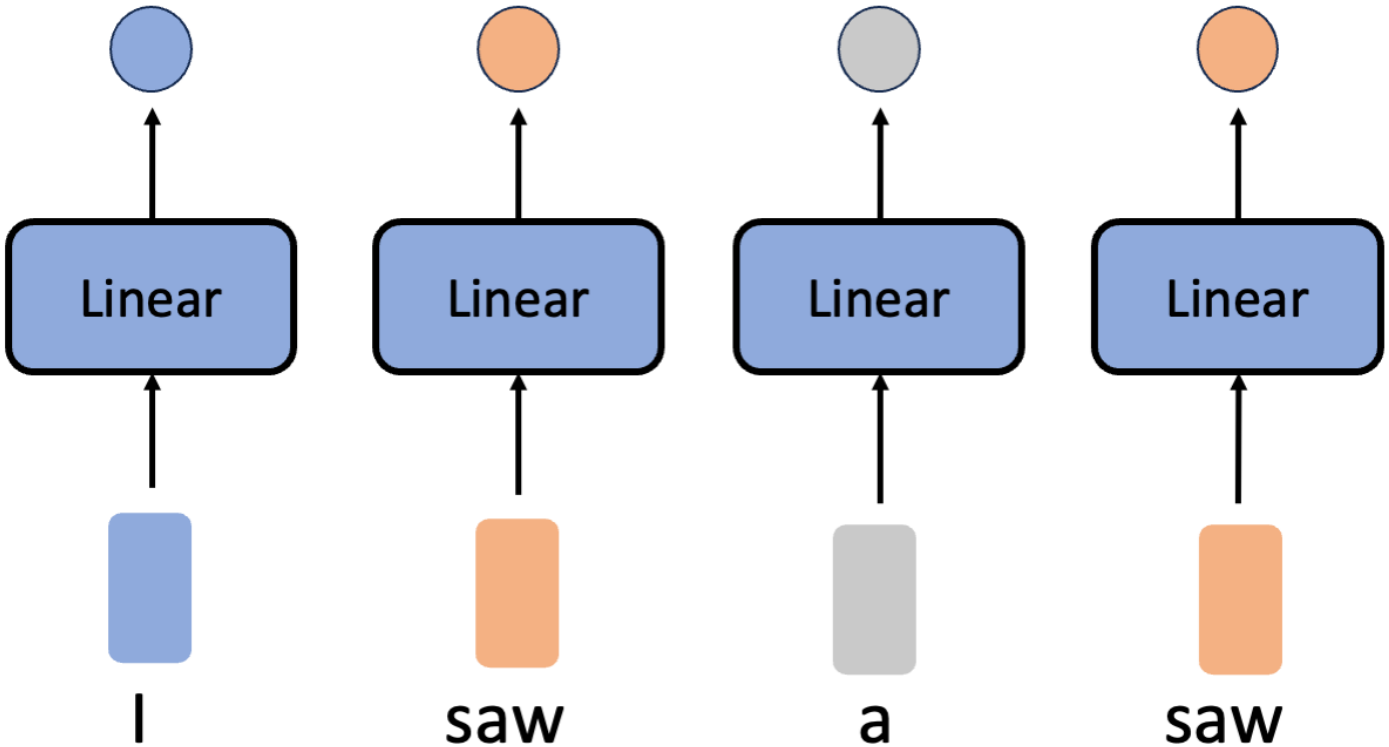
Attention is all you need – Sublayers

- Multi-head Attention (Self-attention / Cross-attention)
- Feed-Forward Networks
- Residual Connections
- Positional Encoding
- Outputs



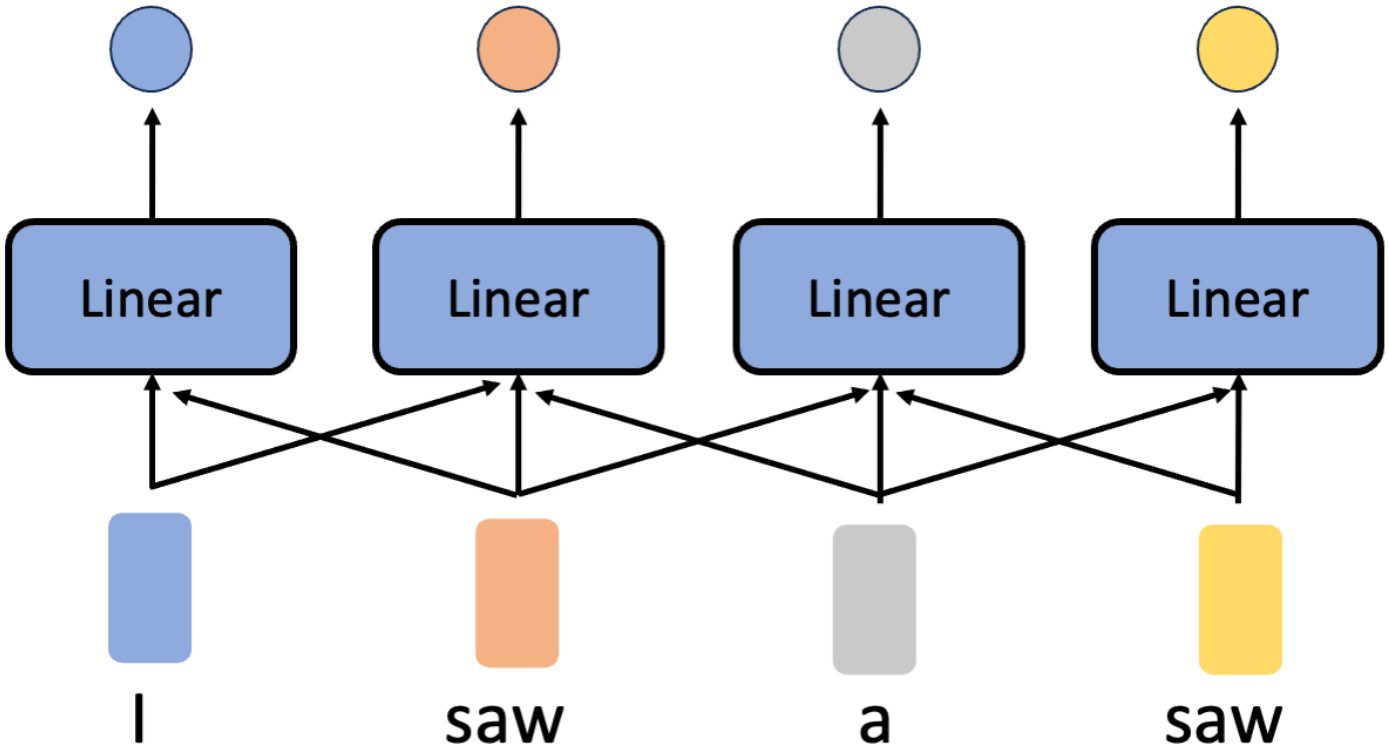
Source: [Vaswani et al., 2017]

Attention is all you need – Recap: Contextual Embeddings



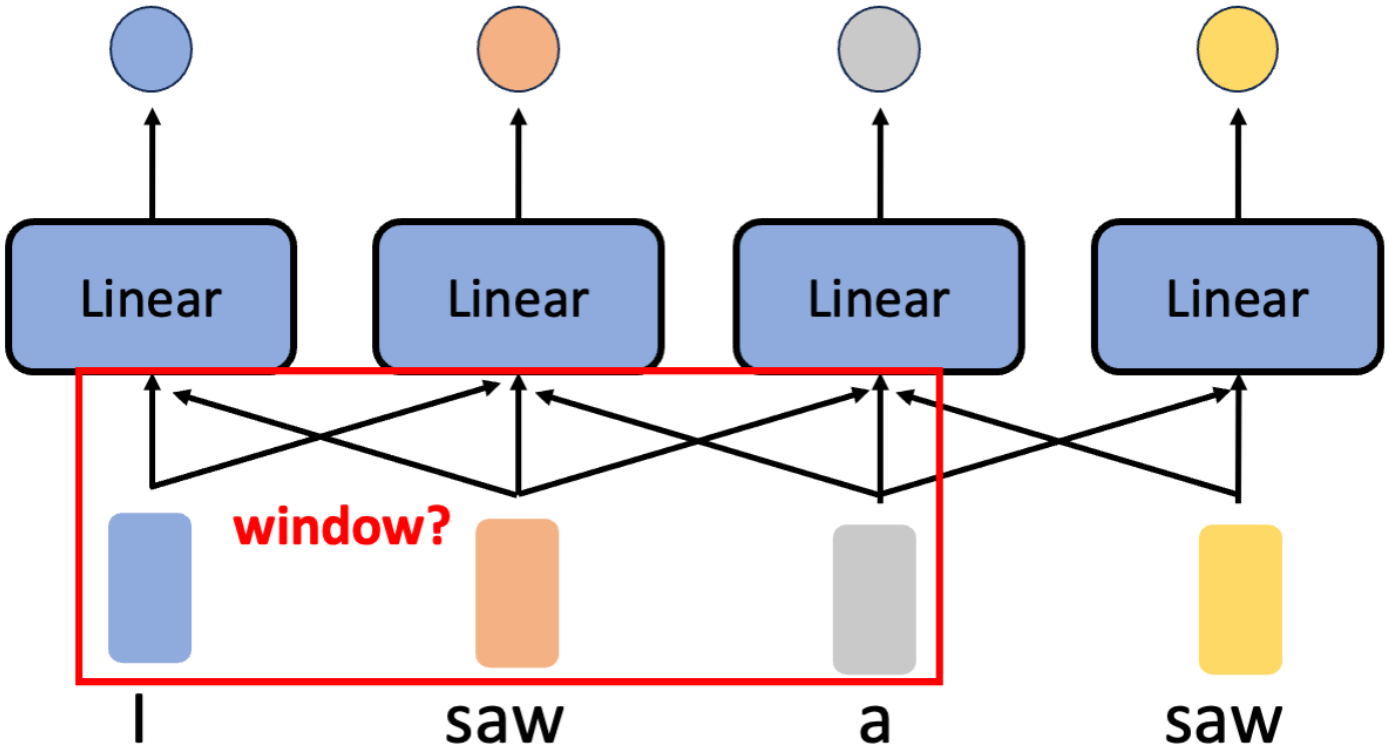
Source: *https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/self_v7.pdf*

Attention is all you need – Recap: Contextual Embeddings



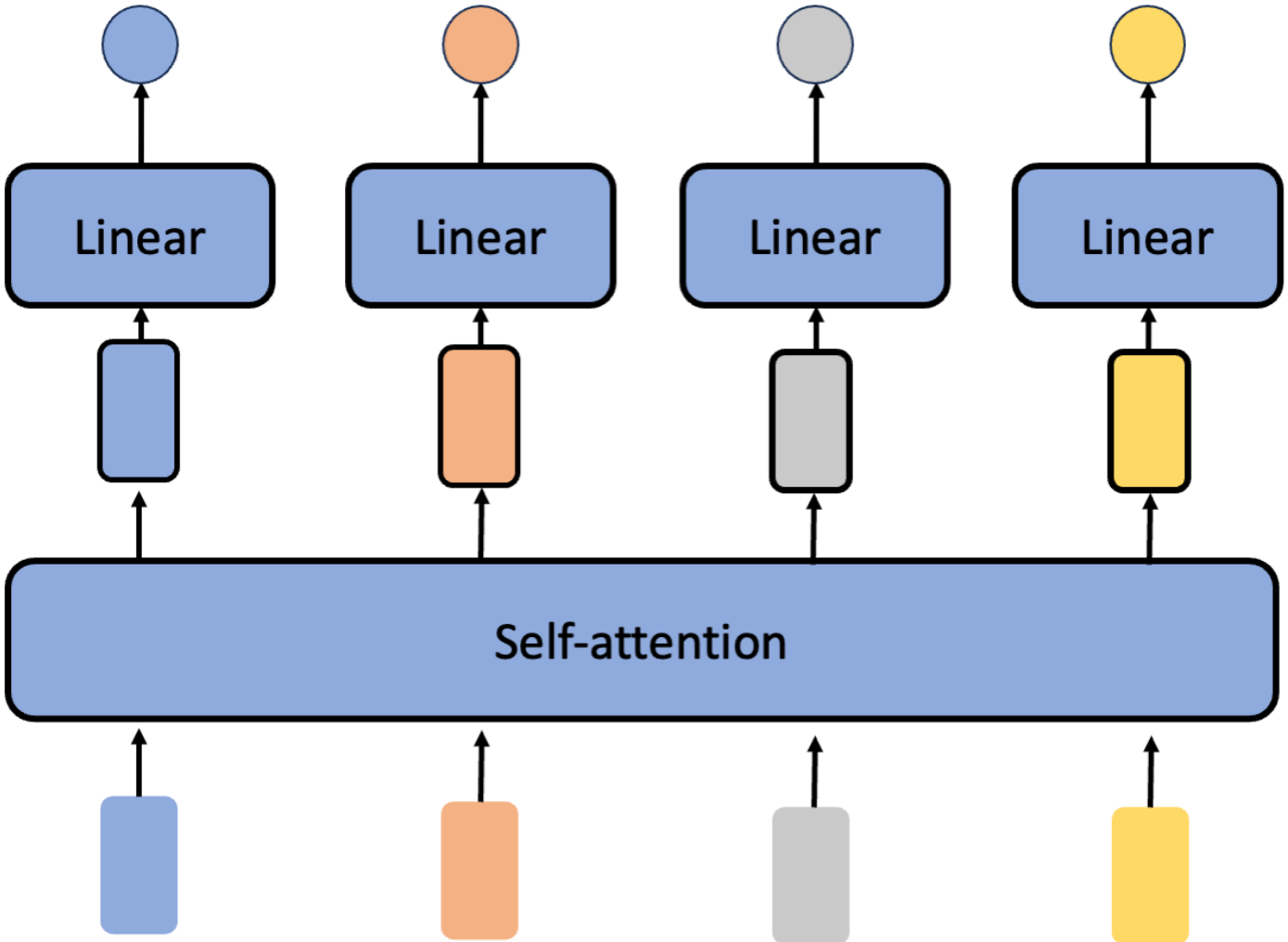
Source: *https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/self_v7.pdf*

Attention is all you need – Recap: Contextual Embeddings



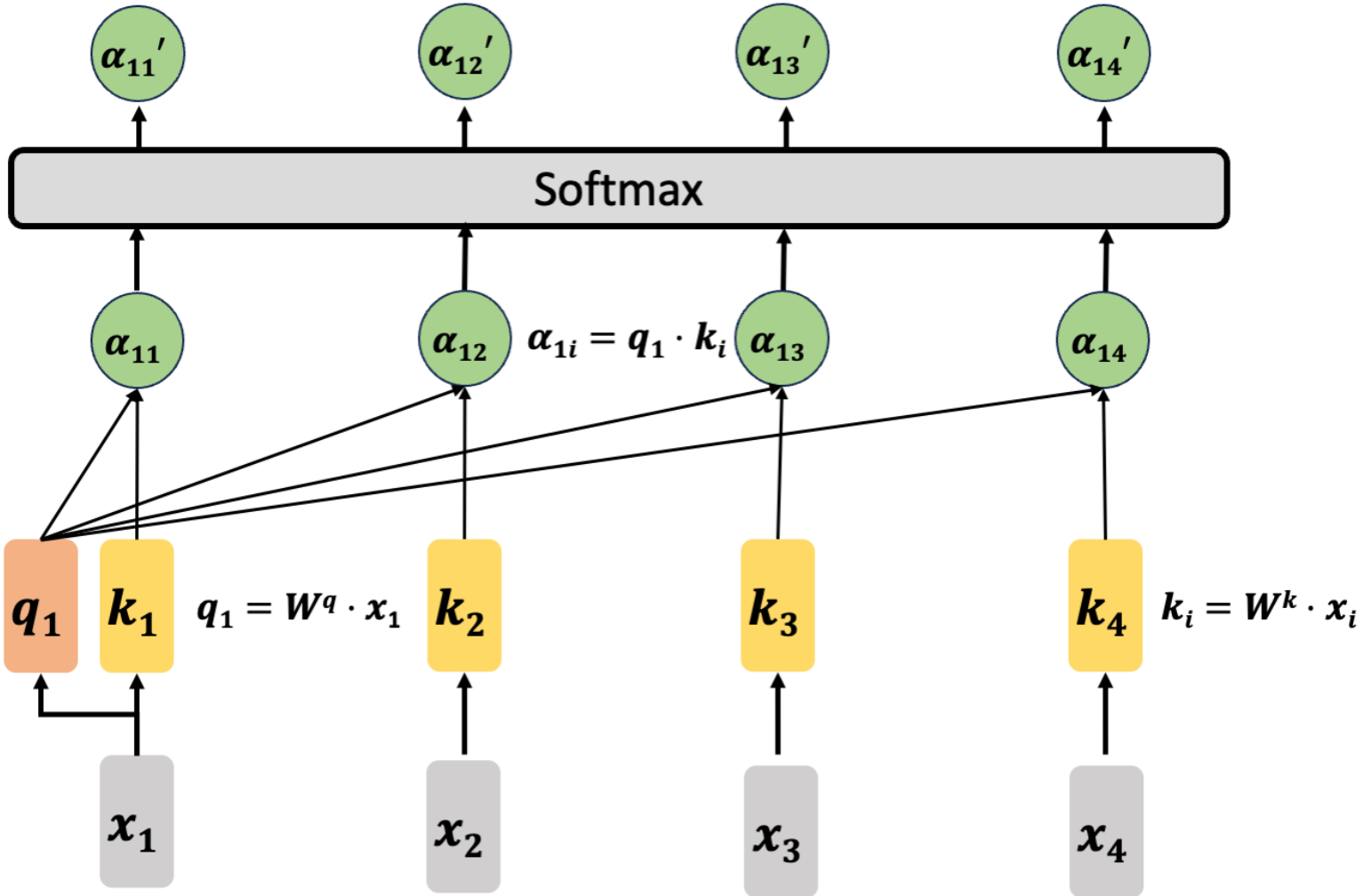
Source: *https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/self_v7.pdf*

Attention is all you need – Contextual Embedding in Self-attention

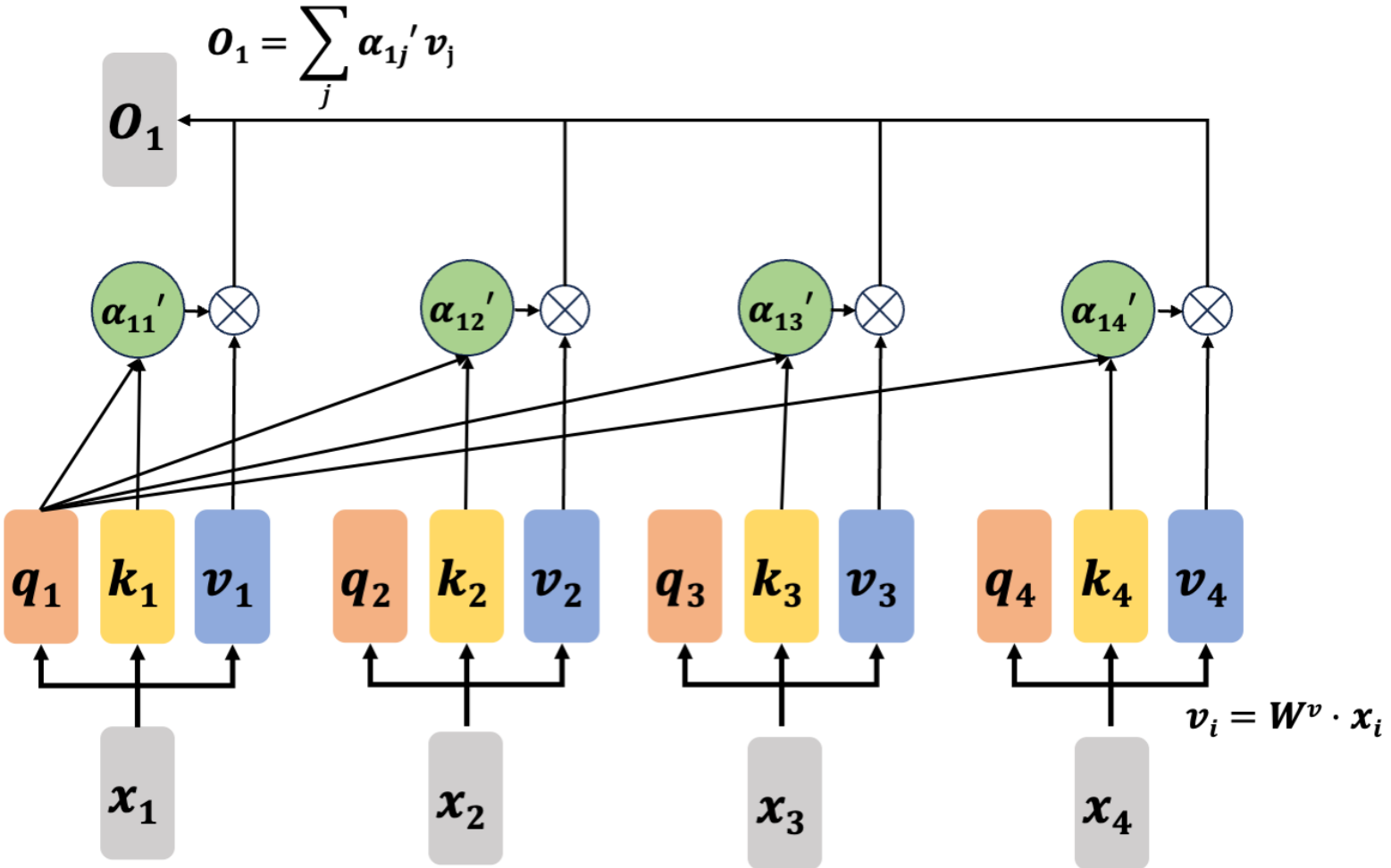


Source: *https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/self_v7.pdf*

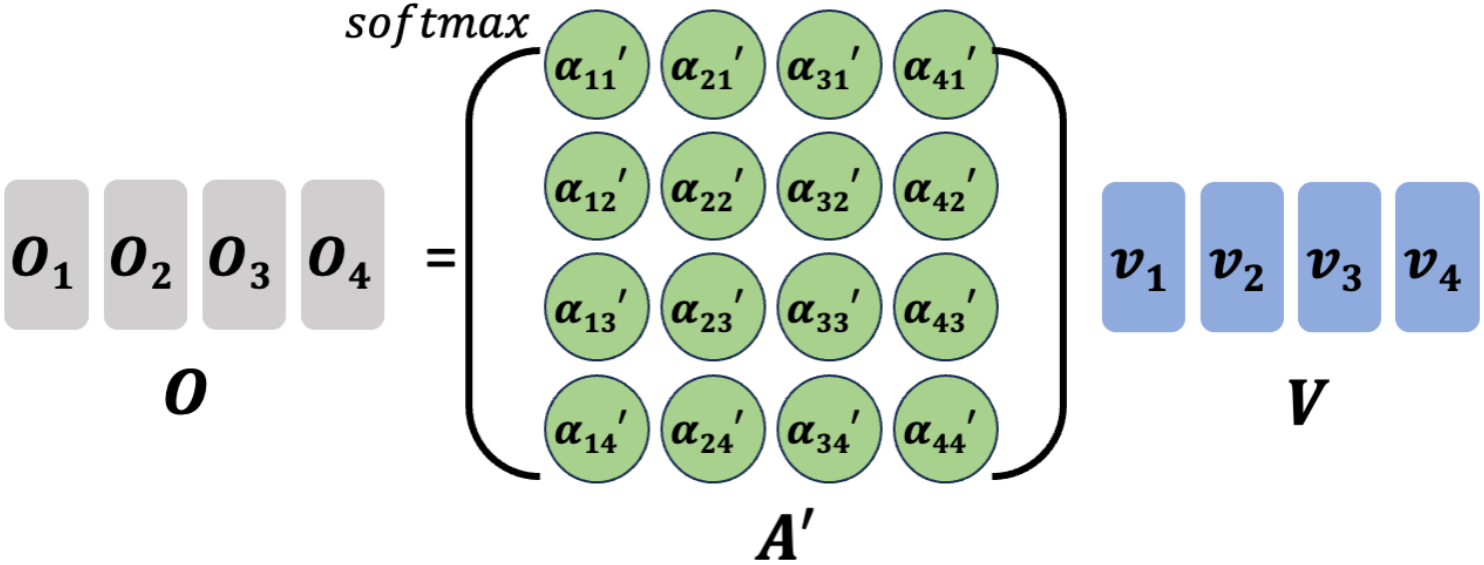
Attention is all you need – Self-attention



Source: *https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/self_v7.pdf*



Source: *https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/self_v7.pdf*

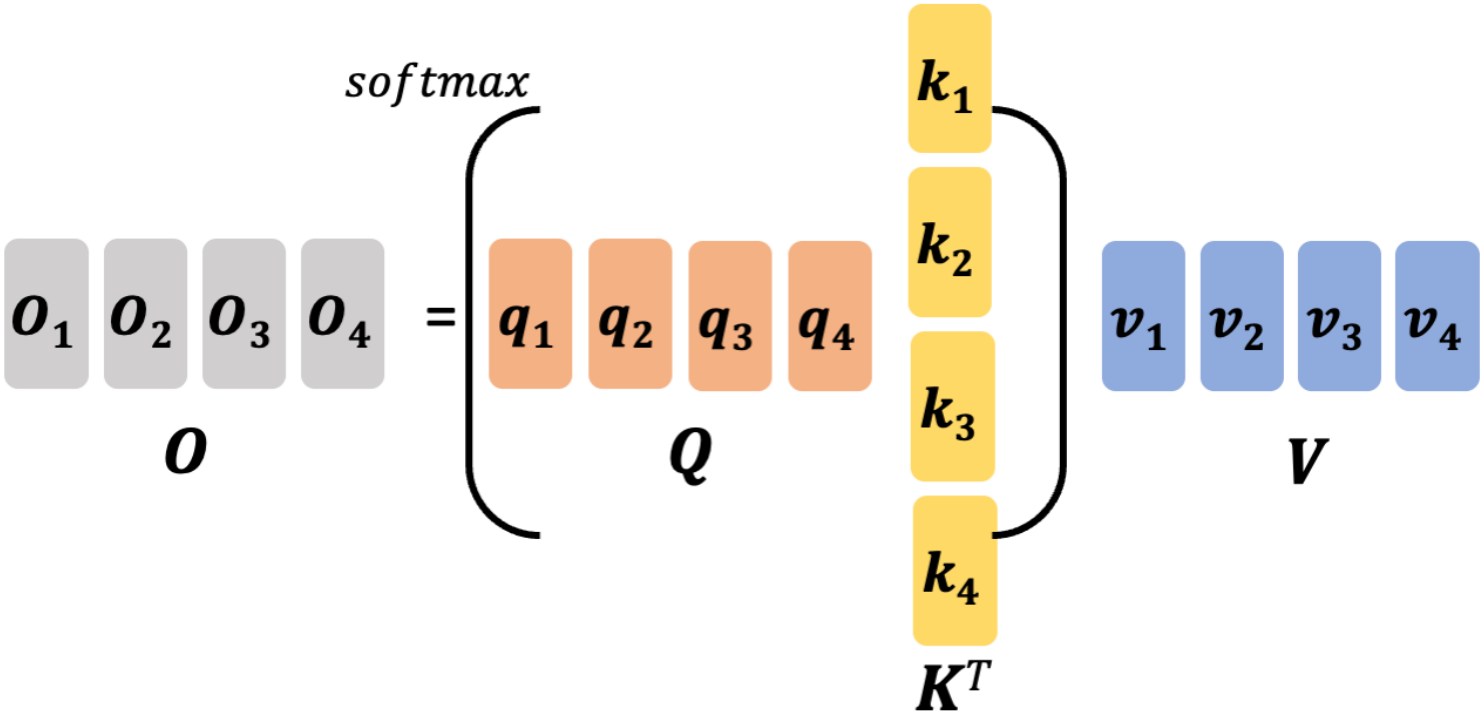


$$o_i = \sum_j \alpha_{ij}' v_j$$

$$\alpha_{i_{j'}} = softmax(q_j \cdot k_i)$$

$$A' = softmax(Q \cdot K^T)$$

Source: *https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/self_v7.pdf*

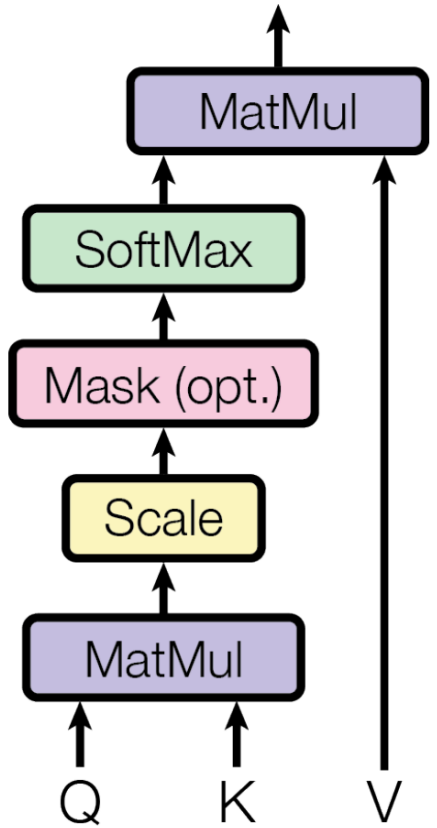


$$O = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

d_k: dimension of input

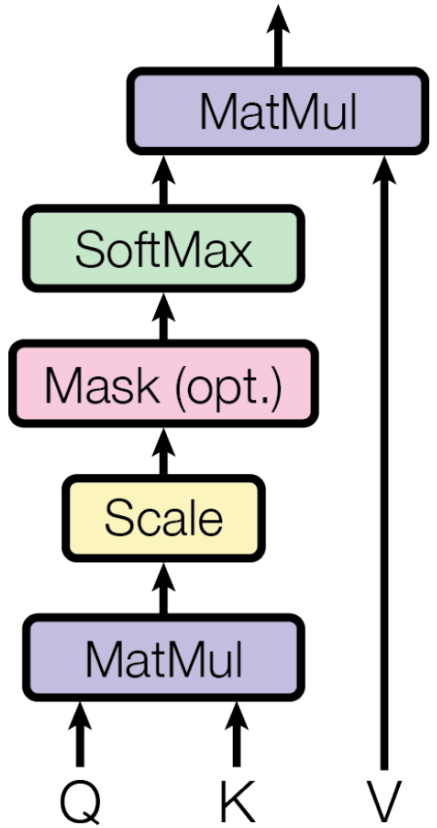
Source: https://speech.ee.ntu.edu.tw/~hylee/ml/ml2021-course-data/self_v7.pdf

- $Attention(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = softmax\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$



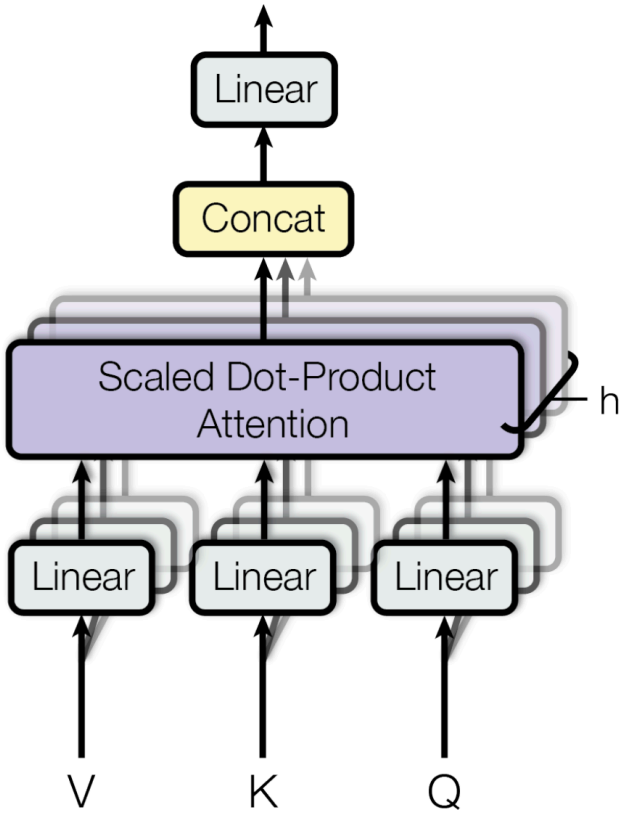
Source: [Vaswani et al., 2017]

- To prevent leftward information flow
- Mask out all illegal (future) connections $\rightarrow$ set numbers in the upper triangle matrix to be $-INF$, then after *softmax* is 0.
- Similar functionality as autoregressive models



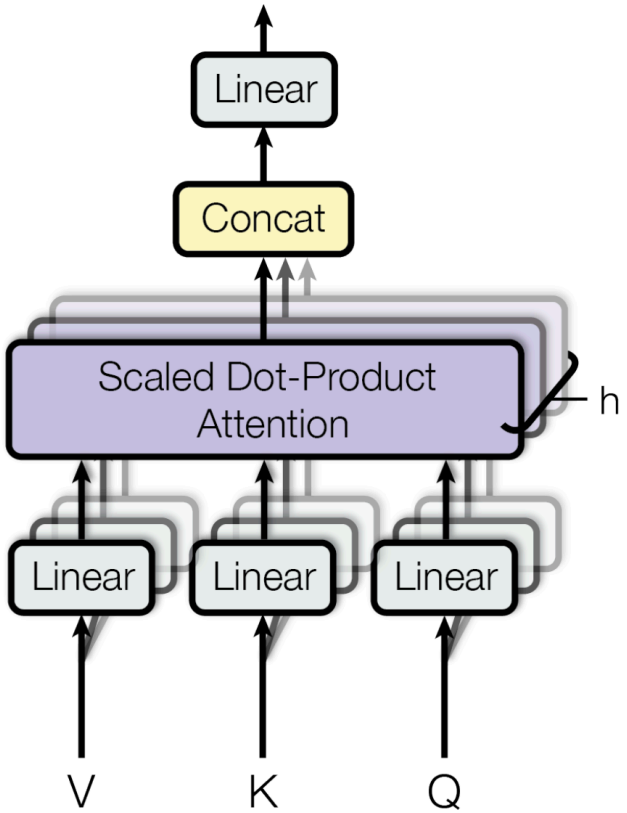
Source: [Vaswani et al., 2017]

- First linearly projects Q, K, V with different, learned linear projections d_q, d_k, d_v



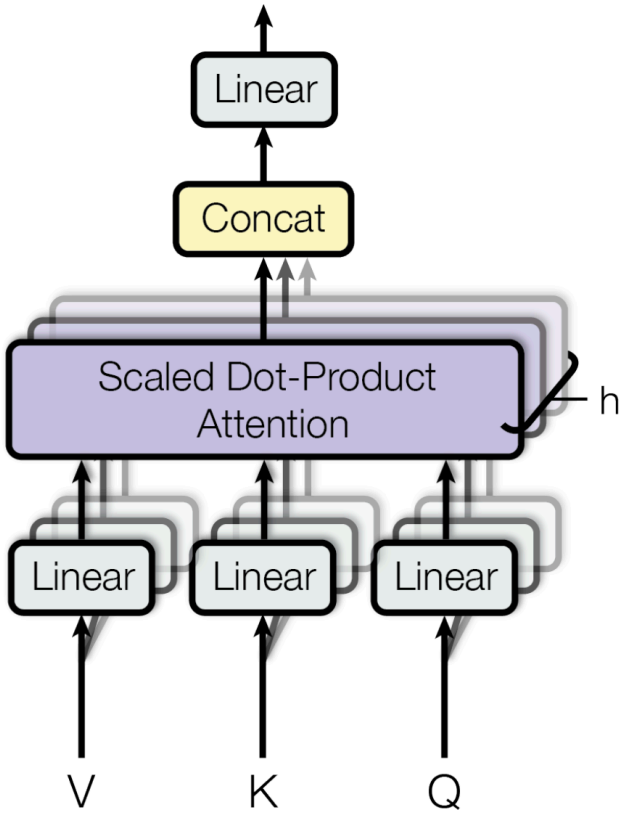
Source: [Vaswani et al., 2017]

- First linearly projects Q, K, V with different, learned linear projections d_q, d_k, d_v
- Done h times on each projected version



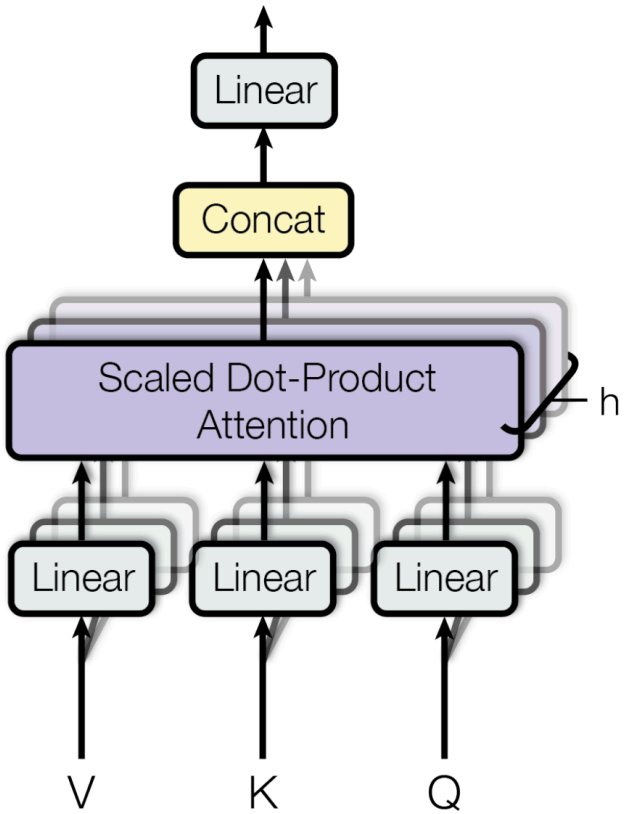
Source: [Vaswani et al., 2017]

- First linearly projects Q, K, V with different, learned linear projections d_q, d_k, d_v
- Done h times on each projected version
- Performs attention function in parallel



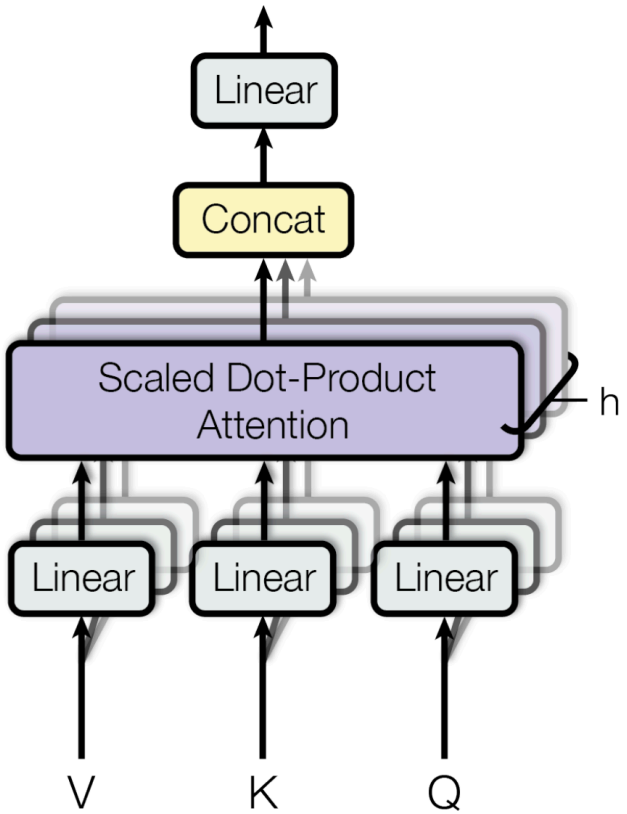
Source: [Vaswani et al., 2017]

- Creates output values of d_v dimensions



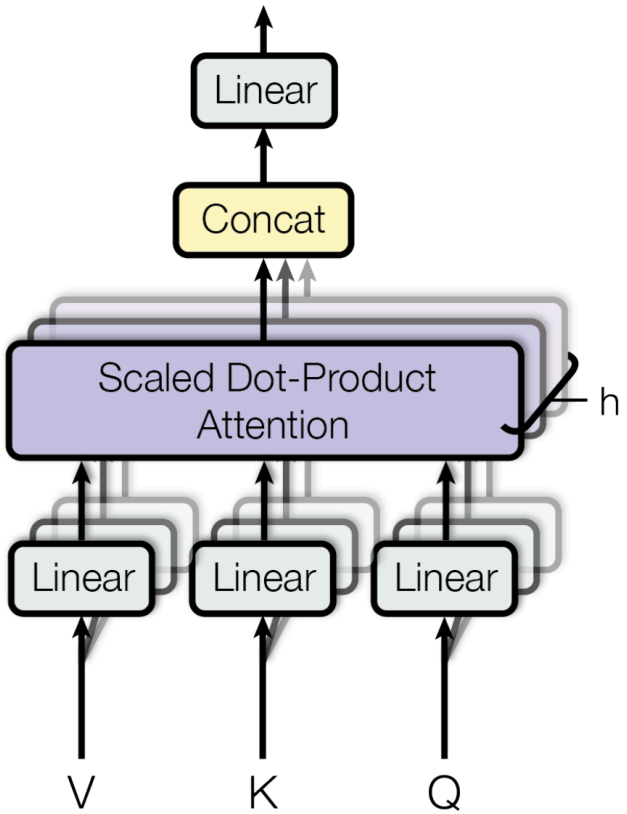
Source: [Vaswani et al., 2017]

- Creates output values of d_v dimensions
- Re-concatenated and re-projected



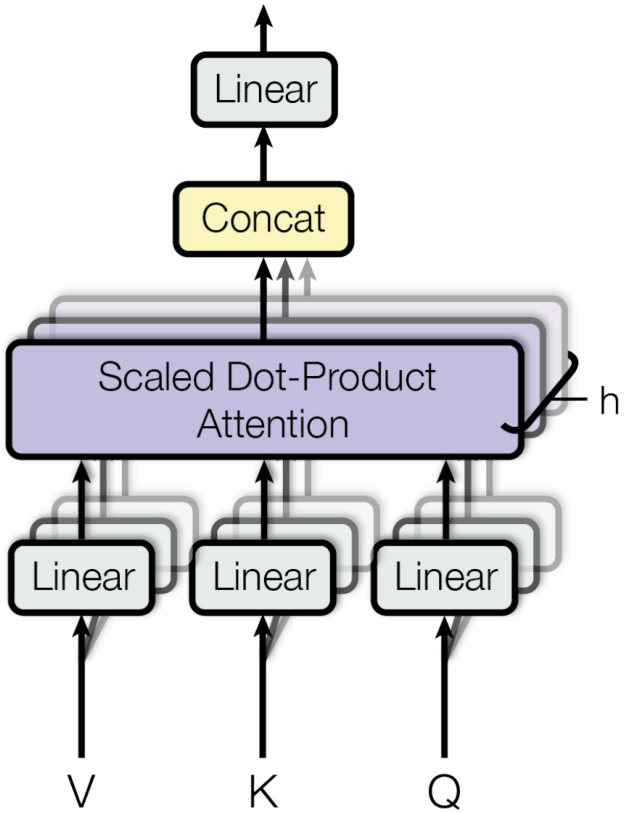
Source: [Vaswani et al., 2017]

- Creates output values of d_v dimensions
- Re-concatenated and re-projected
- This “allows the model to jointly attend to information from different representation subspaces at different positions” [Vaswani et al., 2017].



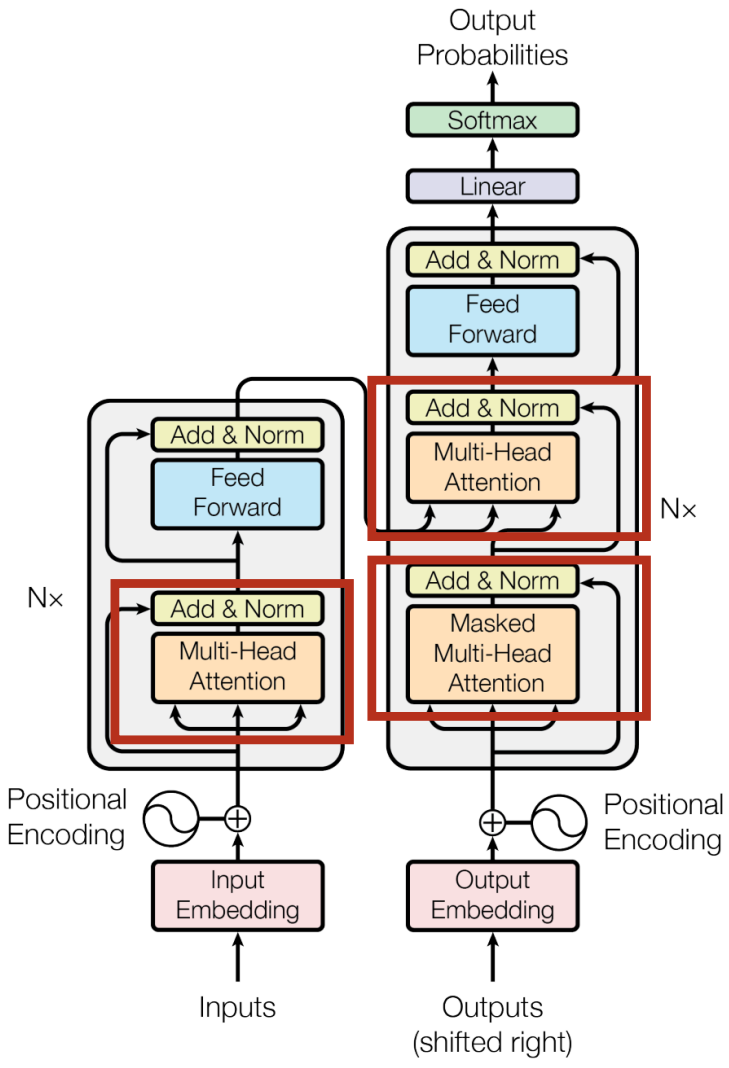
Source: [Vaswani et al., 2017]

- $MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^Q$ where $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$
- where $W_i^Q \in \mathbb{R}^{d_{model} \times d_q}$, $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$ are parameter matrices for the projections



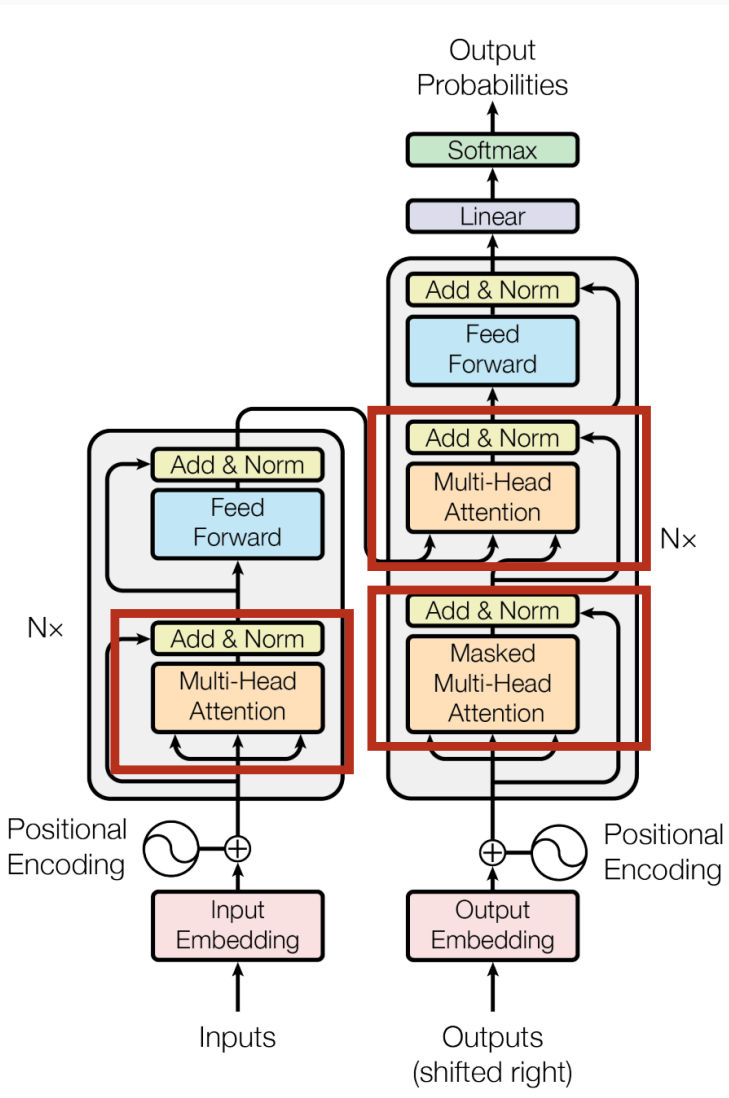
Source: [Vaswani et al., 2017]

- **Self-attention**: Every position of **encoder** and **decoder** attends to every position of previous layer of **encoder** and **decoder**, respectively



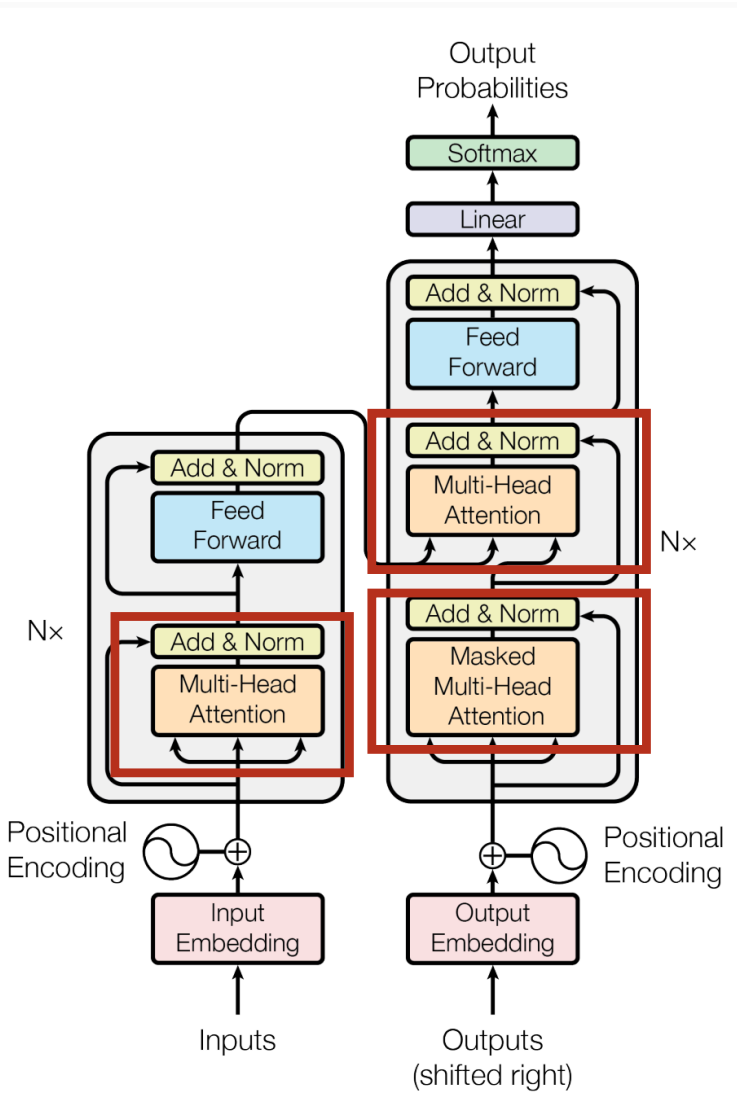
Source: [Vaswani et al., 2017]

- **Self-attention**: Every position of **encoder** and **decoder** attends to every position of previous layer of **encoder** and **decoder**, respectively
- **Cross-attention**: Every position of **decoder** attends to every position of input (**K**, **V** $\rightarrow$ last encoder layer, **Q** $\rightarrow$ the previous decoder layer)



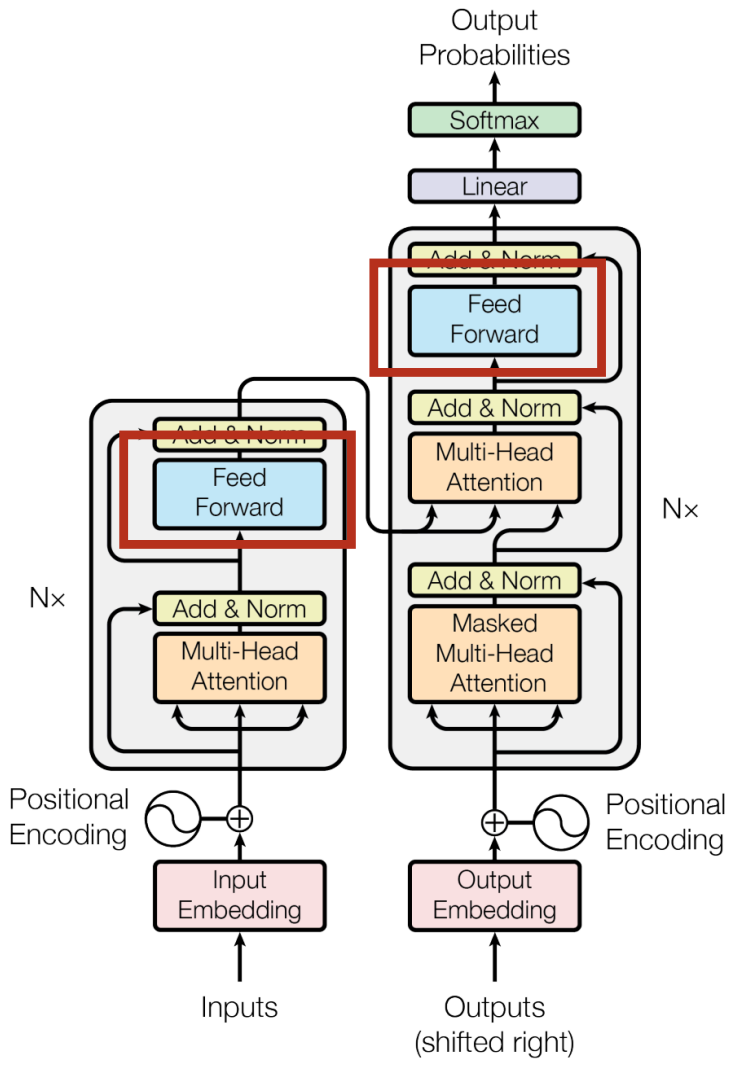
Source: [Vaswani et al., 2017]

- **Self-attention**: Every position of **encoder** and **decoder** attends to every position of previous layer of **encoder** and **decoder**, respectively
- **Cross-attention**: Every position of **decoder** attends to every position of input (**K**, **V** $\rightarrow$ last encoder layer, **Q** $\rightarrow$ the previous decoder layer)
- **Masked Self-attention**: autoregressive functionality



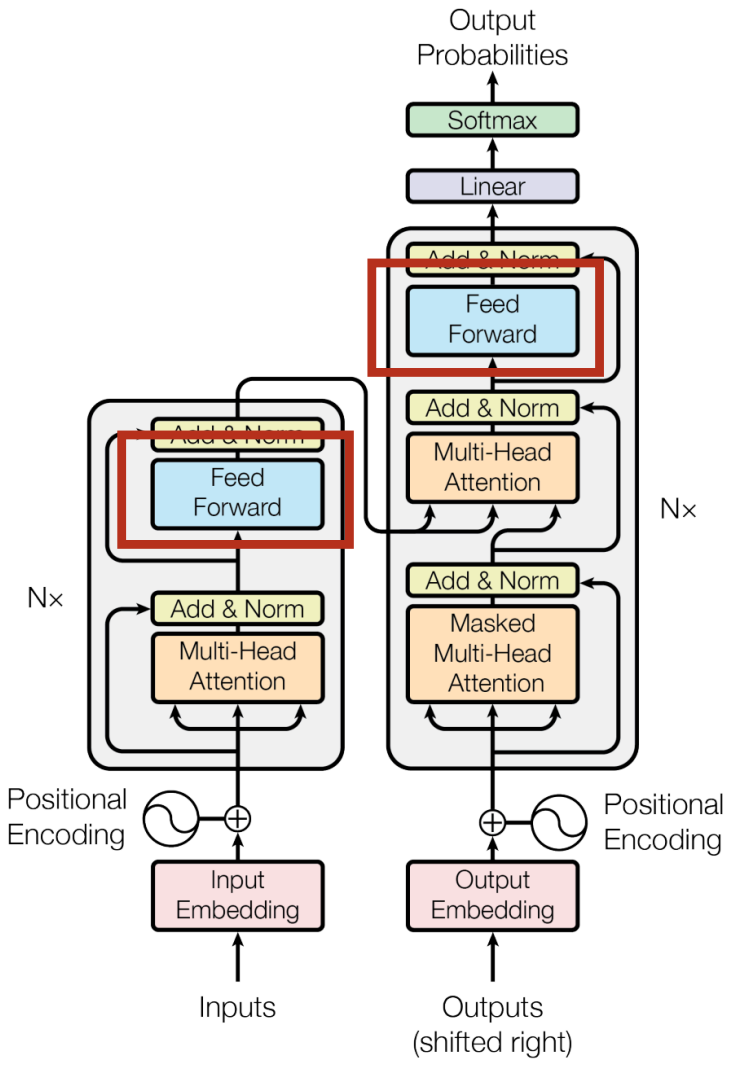
Source: [Vaswani et al., 2017]

- Each FFN has two linear layers (fully-connected) and ReLU activation between them



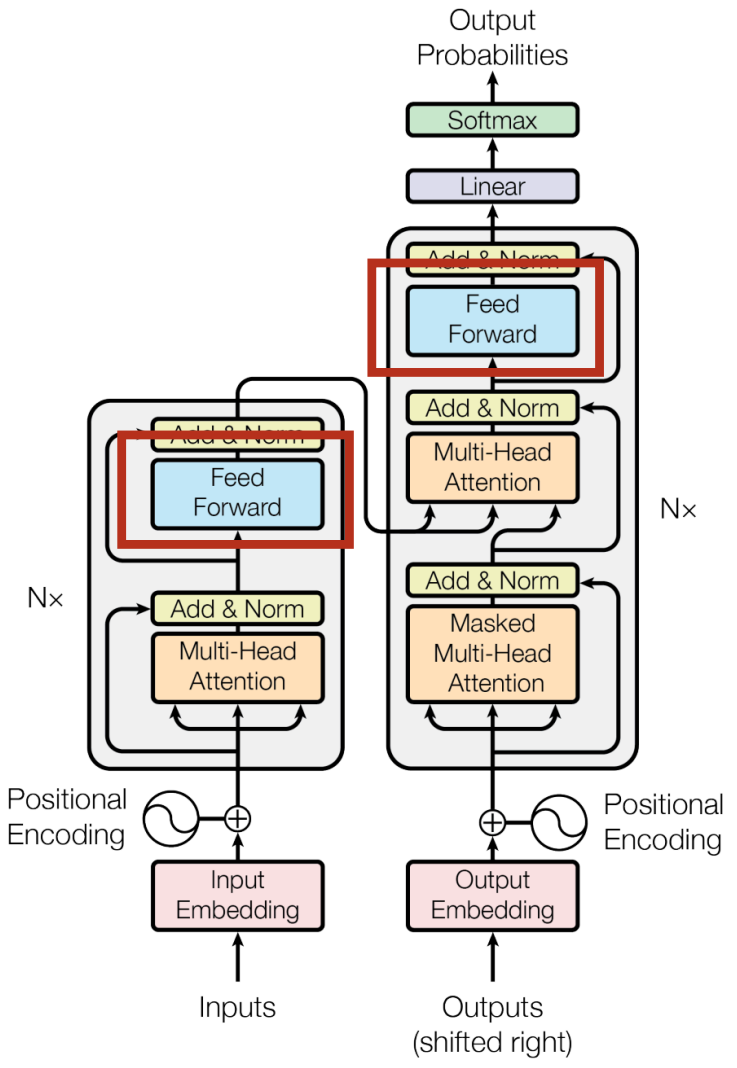
Source: [Vaswani et al., 2017]

- Each FFN has two linear layers (fully-connected) and ReLU activation between them
- Dimension = d_{model} for every linear layer



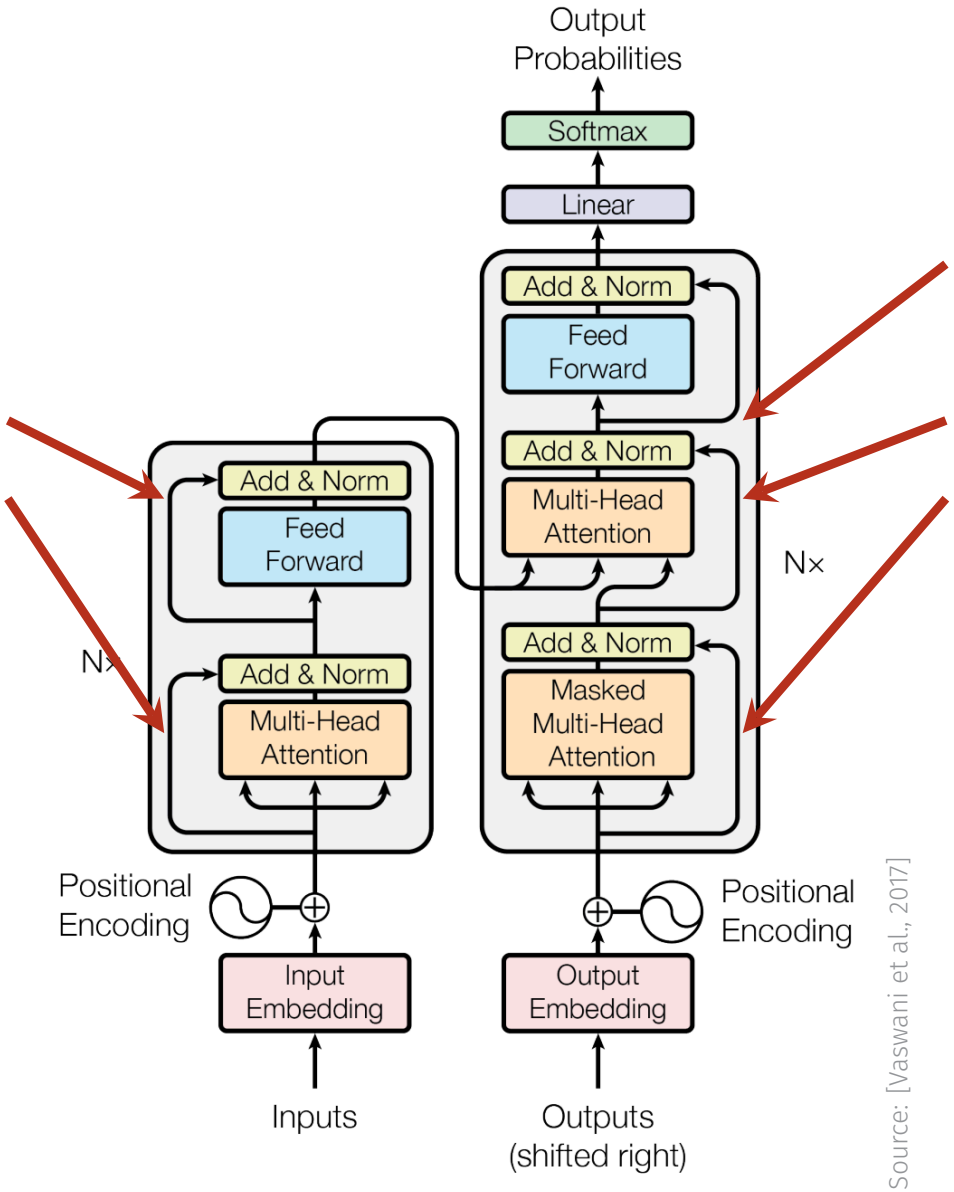
Source: [Vaswani et al., 2017]

- Each FFN has two linear layers (fully-connected) and ReLU activation between them
- Dimension = d_{model} for every linear layer
- $FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$



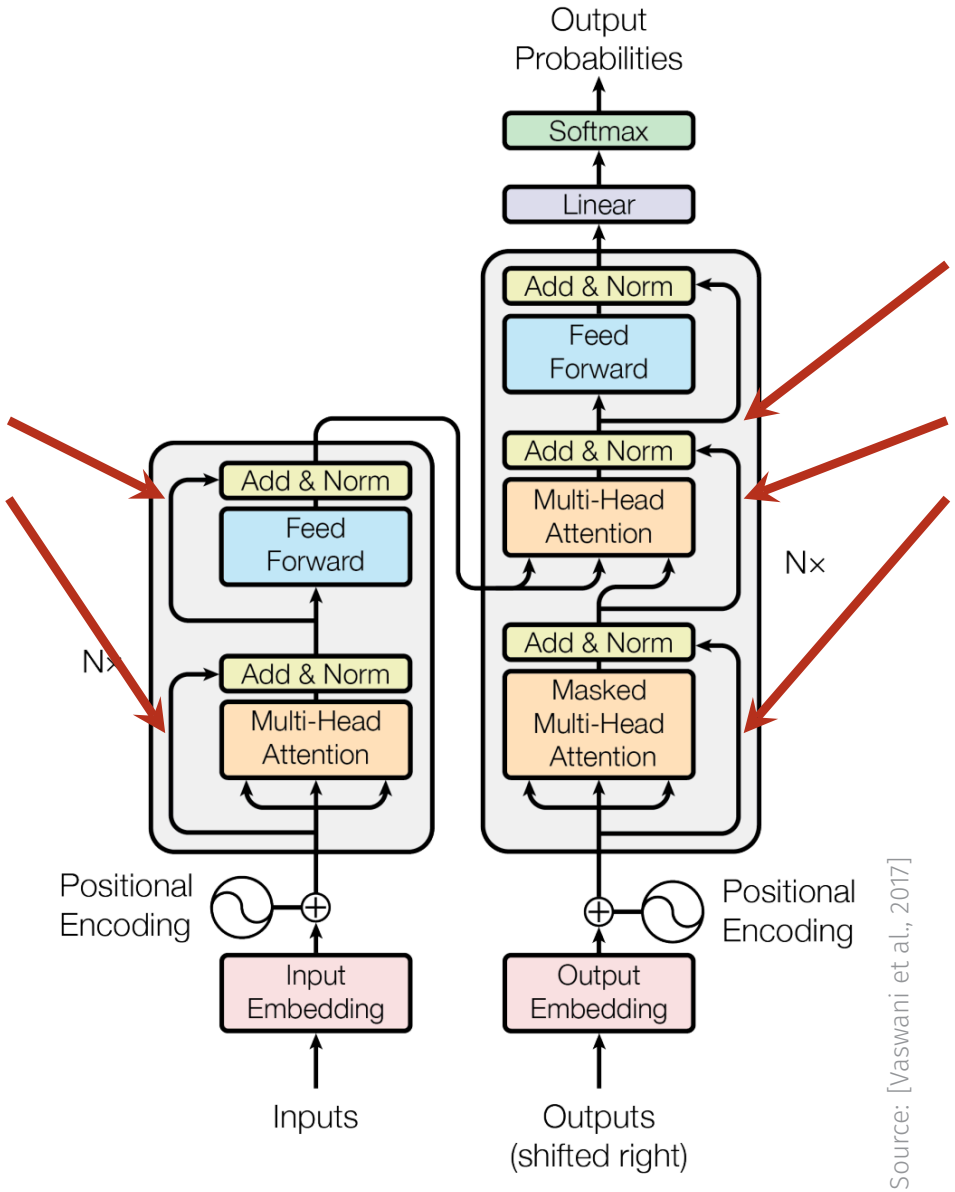
Source: [Vaswani et al., 2017]

- Add: $f(x) + x$
- Does this look familiar?



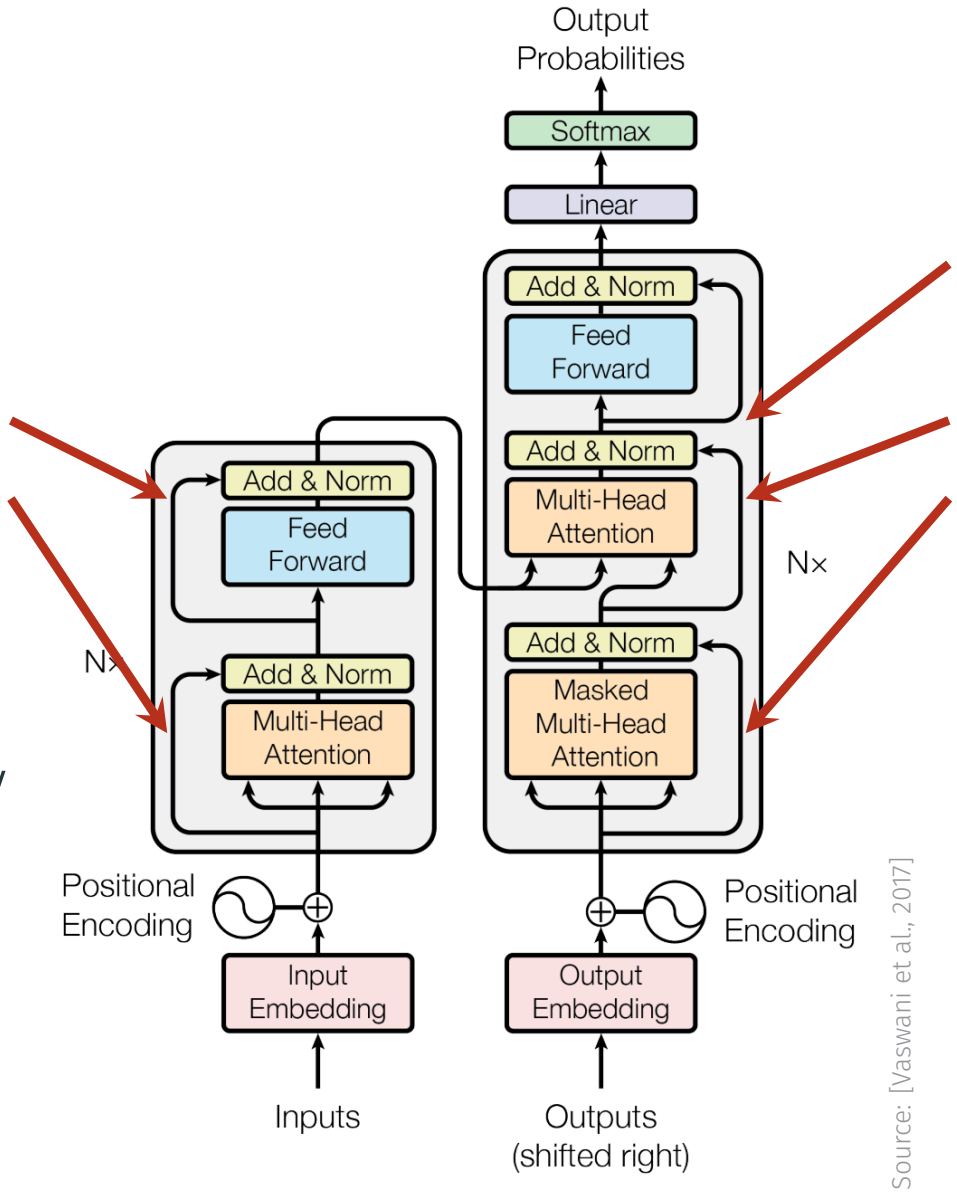
Source: [Vaswani et al., 2017]

- Add: $f(x) + x$
- Does this look familiar?
- Residual! Helps avoid vanishing gradients



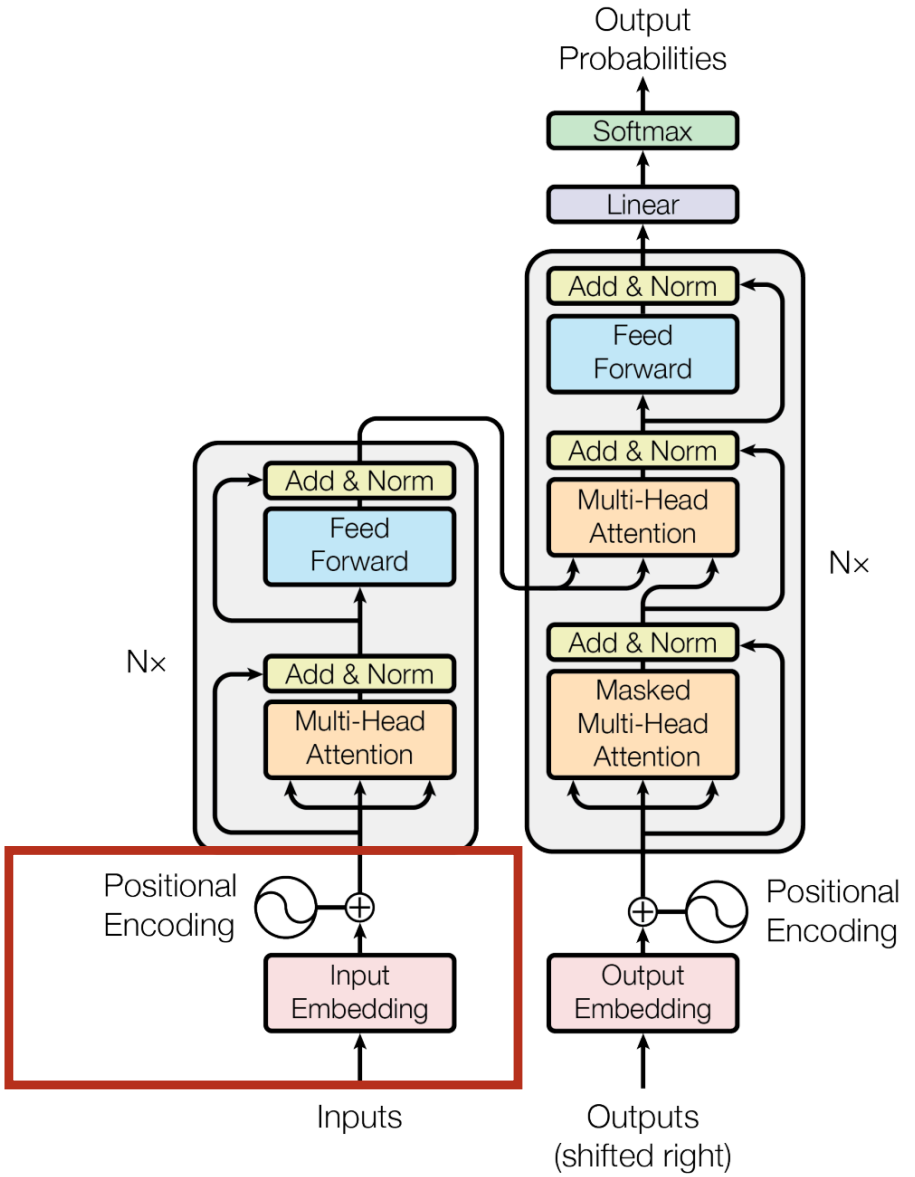
Source: [Vaswani et al., 2017]

- Add: $f(x) + x$
- Does this look familiar?
- Residual! Helps avoid vanishing gradients
- Norm: Layer Normalisation by mean and standard deviation of the values in this layer



Source: [Vaswani et al., 2017]

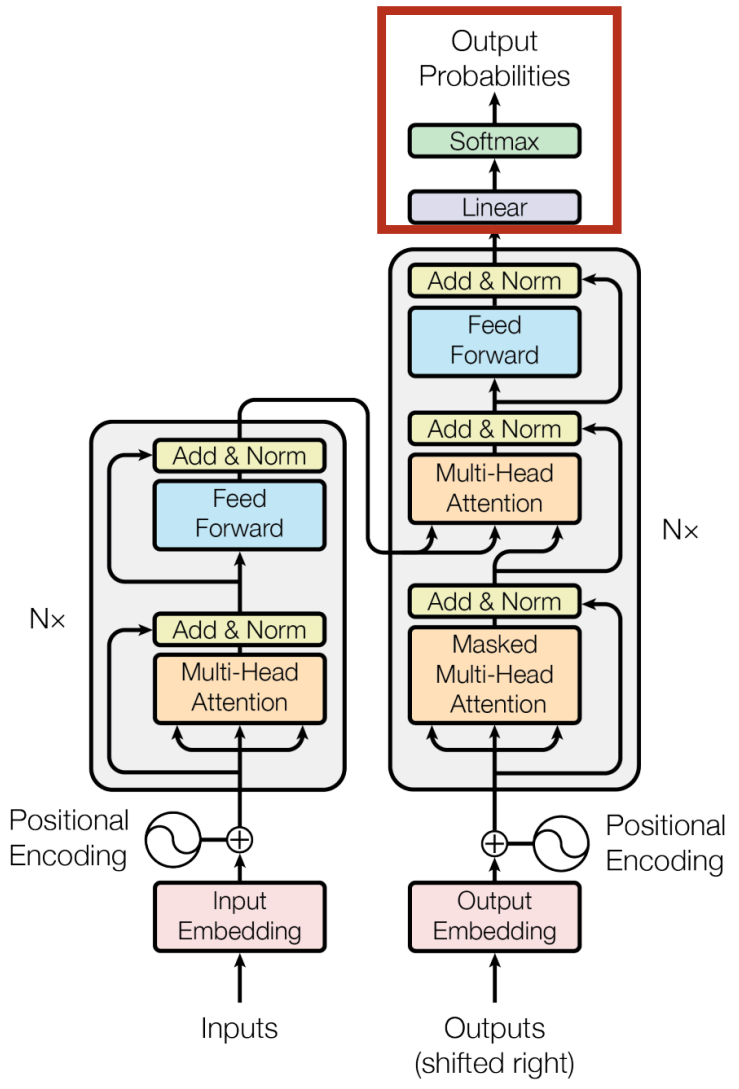
- Inputs = learned embeddings
- Vectors encoded positional biases are added, as the transformer architecture is permutation-invariant
- Dimension = d_{model}



Source: [Vaswani et al., 2017]

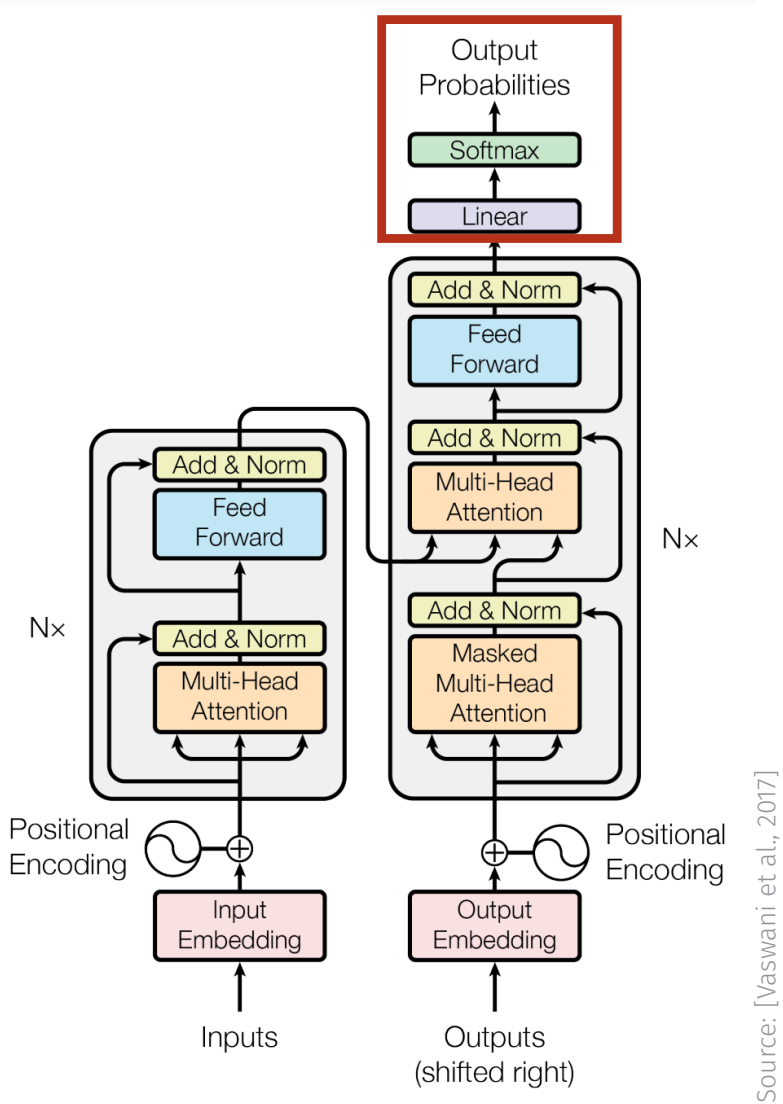
Attention is all you need – Outputs

- Use learned linear transformations & softmax for output



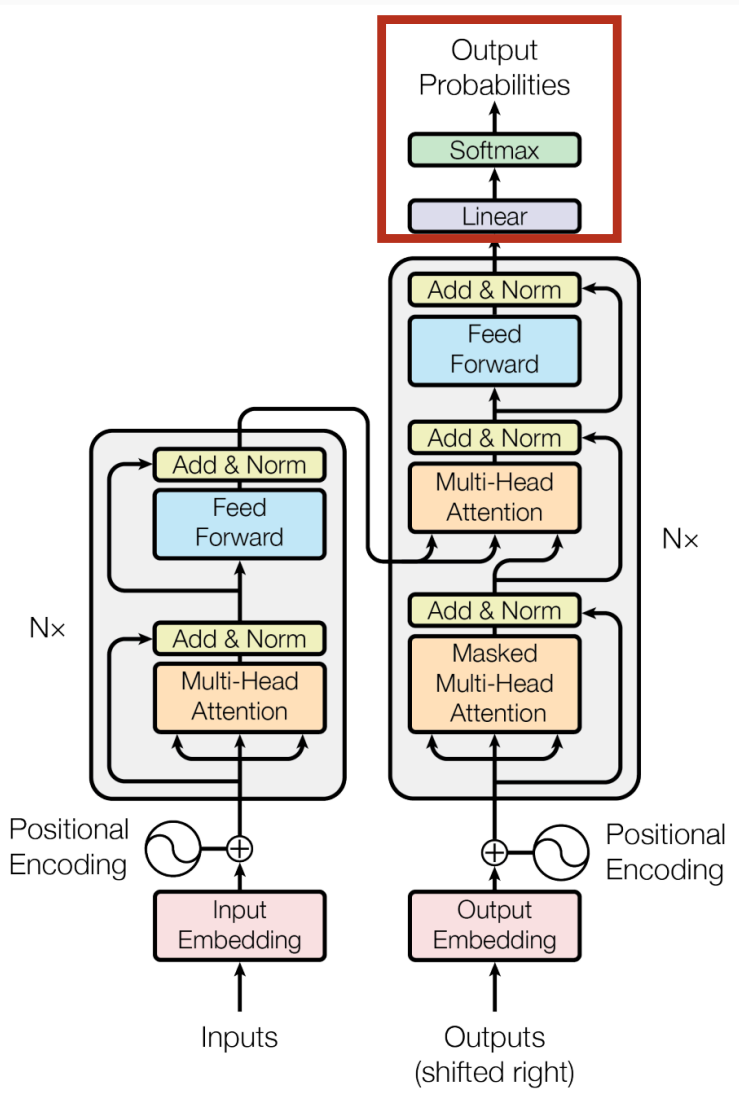
Source: [Vaswani et al., 2017]

- Use learned linear transformations & softmax for output
- Converts output to probabilities for next token



Source: [Vaswani et al., 2017]

- Use learned linear transformations & softmax for output
- Converts output to probabilities for next token
- Weights of embeddings are multiplied by $\sqrt{d_{model}}$

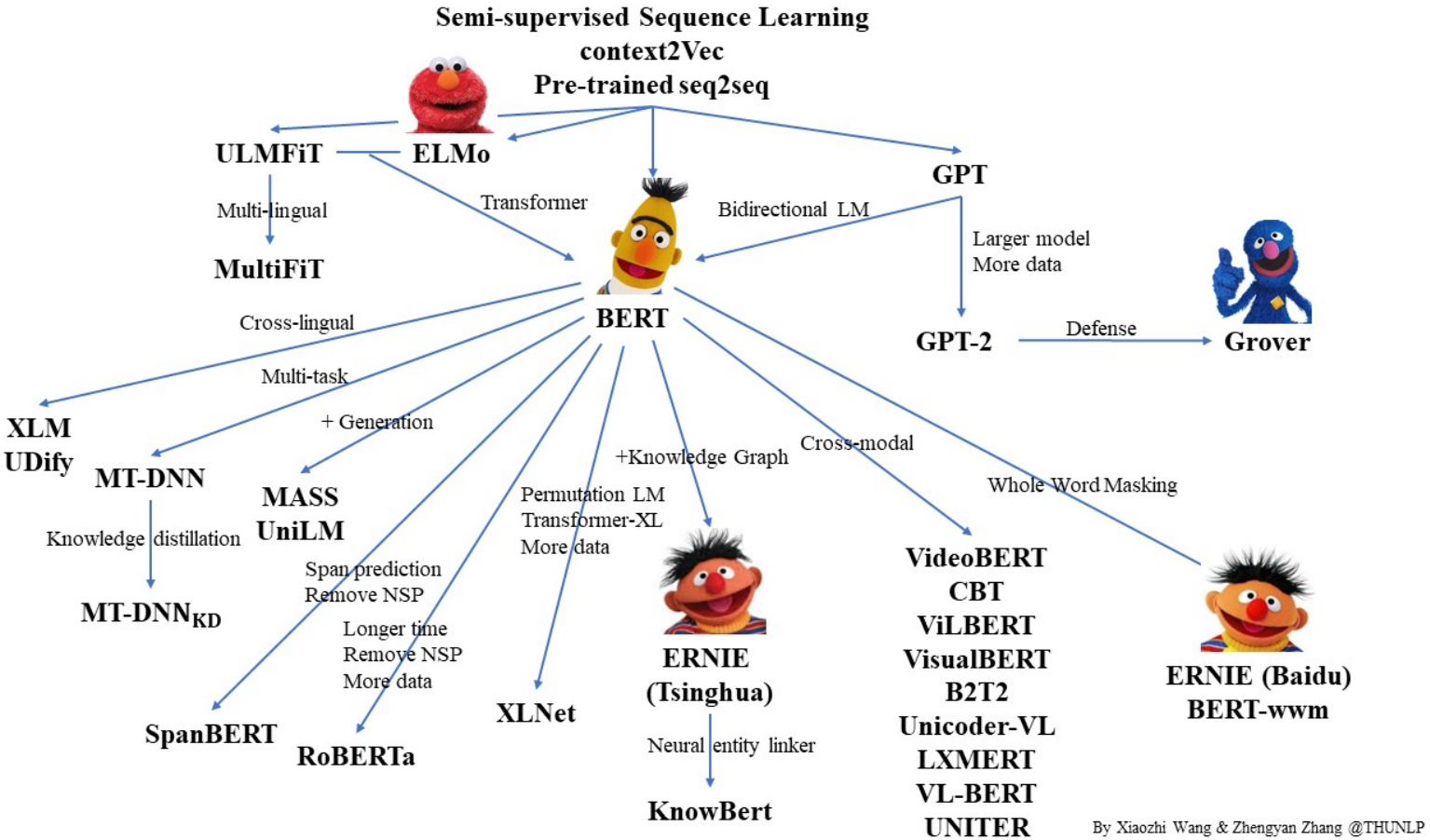


Source: [Vaswani et al., 2017]

Transformers

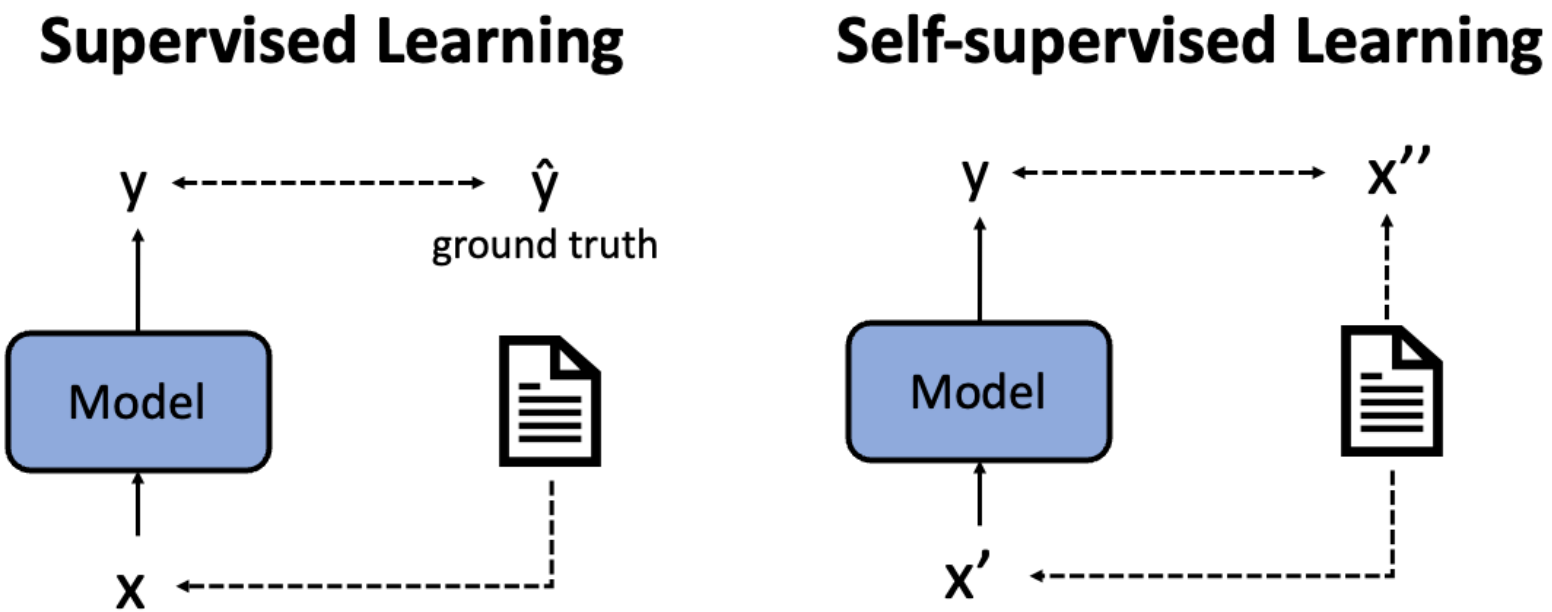
BERT

- ACL 19': Bidirectional Encoder Representations from Transformers [Devlin, Chang, Lee, and Toutanova, 2019]
- BERT-base: 12 layers, 12 attention heads, 110 million parameters
- BERT-large: 24 layers, 16 attention heads, 340 million parameters

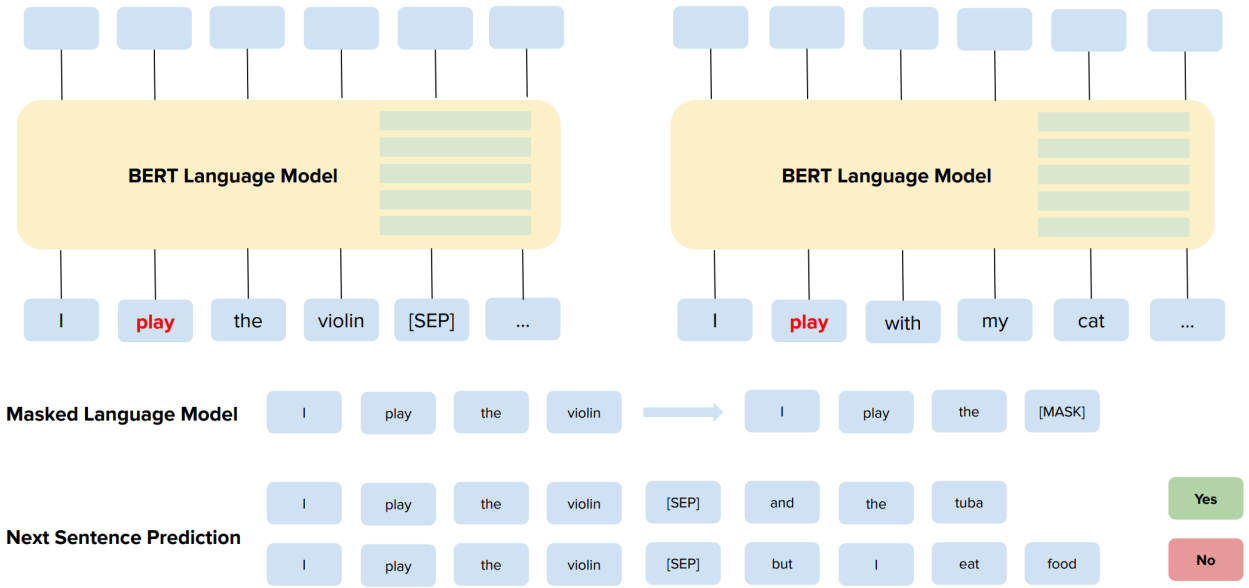


Source: THUNLP, *http://nlp.csai.tsinghua.edu.cn/*

- Mask random 15% of tokens
  - 80% - replace word by [MASK]
  - 10% - replace word by random token
  - 10% - don’t replace word
- Trained on unlabeled data (**self-supervised learning**), fine-tuned on task-specific, labeled data


Supervised Learning
y
ŷ
ground truth
Model
x
Self-supervised Learning
y
x''
Model
x'

BERT – Training



Source: Nozza et al. [2020]

- Word prediction (Masked Language Model, MLM)
- Next sentence prediction: Given two sentences, predict if this is the next one (binary classification)

BERT – Training

- Trained on BooksCorpus and English Wikipedia
- Took 4 days on 64 TPU chips!
- Greatly improved SOTA in downstream tasks
- Pre-trained model was released
- Many improved variations such as RoBERTa and ModernBERT

Transformers – Summary of Today's Lecture

- Contextual embeddings (e.g. Bert) are better representations than non-contextual embeddings (e.g. Word2Vec)
- Attention mechanism:
  - soft vs. hard attention
  - local vs. global attention
  - self- vs. cross-attention
- Transformer architecture, SOTA transformers such as Bert (actually now ViLTs...)
- Training transformers takes a LOT of training data, GPU memory, and time (advantage of RNNs)

Transformers – Further Reading

- Soft and hard attention
- Illustrated Transformer
- Transformer Explainer
- Long Range Arena: A Benchmark for Efficient Transformers
- Attention in Transformers (Youtube Video)

Transformers in Computer Vision

- End-to-end Object Detection in Visual Transformers
- Self-supervised Vision Transformers

References i

J. Ba, V. Mnih, and K. Kavukcuoglu. Multiple object recognition with visual attention. *arXiv preprint arXiv:1412.7755*, 2014.

D. Bahdanau, K. Cho, and Y. Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.

J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*, 2019.

K. Ethayarajh. How contextual are contextualized word representations? comparing the geometry of bert, elmo, and gpt-2 embeddings. *arXiv preprint arXiv:1909.00512*, 2019.

J. Firth. *Studies in linguistic analysis*. Blackwell, Oxford, 1957.

G. W. Lindsay. Attention in Psychology, Neuroscience, and Machine Learning. *Frontiers in Computational Neuroscience*, 14:29, 2020.

M.-T. Luong, H. Pham, and C. D. Manning. Effective Approaches to Attention-based Neural Machine Translation. *arXiv preprint arXiv:1508.04025*, 2015.

T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*, 2013a.

T. Mikolov, Q. V. Le, and I. Sutskever. Exploiting Similarities among Languages for Machine Translation. *arXiv preprint arXiv:1309.4168*, 2013b.

V. Mnih, N. Heess, A. Graves, et al. Recurrent models of visual attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2204–2212, 2014.

D. Nozza, F. Bianchi, and D. Hovy. What the [MASK]? Making Sense of Language-Specific BERT Models. *arXiv preprint arXiv:2003.02912*, 2020.

J. Pennington, R. Socher, and C. Manning. Glove: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, t. Kaiser, and I. Polosukhin. Attention is All you Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.

K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhudinov, R. Zemel, and Y. Bengio. Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. pages 2048–2057, 2015.